# EE5907 Pattern Recognition

## Lecture 3 Linear and Non-linear Classifiers

© Copyright National University of Singapore. All Rights Reserved.

# Discussion on Tutorial: Q1

- Show that in a multiclass problem with $M$ classes, the probability of classification error for the optimal classifier is bound by:

$$p_{err} \leq \frac{M-1}{M}$$

© Copyright National University of Singapore. All Rights Reserved.

# Discussion on Tutorial: Q2

- Suppose that we have a random variable $x$ which follows an Erlang distribution as: $p(x|\theta) = \theta^2 x \exp(-\theta x)\, u(x)$ where $u(x)$ is the unit step function defined as:

$$u(x) = \begin{cases} 1 & if\ x > 0 \\ 0 & if\ x \leq 0 \end{cases}$$

- Now show that the Maximum Likelihood Estimation (MLE) of $\theta$ given $N$ measurements $x_1, \dots, x_N$ of $x$ is defined as:

$$\hat{\theta}_{MLE} = \frac{2N}{\sum_{k=1}^{N} x_k}$$

© Copyright National University of Singapore. All Rights Reserved.

# Discussion on Tutorial: Q3

- (a) Suppose a crime has been committed. Blood is found at the scene for which there is no innocent explanation. The blood of a type which is present in 1% of the population. The prosecutor claims: “There is a 1% chance that the defendant would have the crime blood type if he were innocent. Thus, there is a 99% chance that he is guilty”. This is known as the prosecutor’s fallacy. Explain in detail what is wrong with this argument.

© Copyright National University of Singapore. All Rights Reserved.

# Discussion on Tutorial: Q3

- (b) Further, the defender claims: “The crime occurred in a city of 800,000 people. The blood type would be found in approximately 8000 people. The evidence has provided a probability of just 1 in 8000 that the defendant is guilty and thus has no relevance.” This is known as the defender’s fallacy. Explain in detail what is wrong with this argument?

© Copyright National University of Singapore. All Rights Reserved.

# Linear Classifiers:

- ✓ Perceptron Algorithm and its Variants.
- ✓ Least Square Methods for Optimal Linear Classifiers.
- ✓ The Bias-Variance Dilemma.

© Copyright National University of Singapore. All Rights Reserved.

# Linear Classifiers: Simple yet Important

- Intuitively, what is the advantage(s) of linear classifiers?
  - Simplicity and computational attractiveness.
  - Of course, linear classifiers are NOT perfect.
- First assume that *all* feature vectors from the available classes can be classified correctly using a linear classifier.
  - The goal: design an “optimal linear classifier” by adopting an appropriate optimality criterion.
  - Further assumptions (more general case) will be that feature vectors may not be classified correctly – the risk of mis-classification.

© Copyright National University of Singapore. All Rights Reserved.

# Discriminant Functions and Decision Surfaces

- **Bayes decision rule**: $\pi^*(x) = \arg\min_{a \in \mathcal{A}} \rho(a|x) = \arg\max_{y \in \mathcal{Y}} p(y|x)$
  - Equivalent to partitioning the feature space into $C$ regions, for a classification task with $C$ classes.
  - First focus the **binary classification** case: $\mathcal{Y} = \{1,2\}$, thus the partition will result in 2 regions.
  - We can safely assume that the two regions are contiguous – they are thus separated by a "**decision surface**" (also known as a **decision hypersurface**) in the feature space.
  - The decision surface is characterized by the following equation:

$$p(y_1|x) - p(y_2|x) = 0$$

© Copyright National University of Singapore. All Rights Reserved.

# Discriminant Functions and Decision Surfaces

$$p(y_1|x) - p(y_2|x) = 0$$

- From one side of the surface the difference $> 0$ and vice versa;
- Often, more convenient to work with an equivalent function:
  - $g_i(x) \equiv f\big(p(y_i|x)\big)$. $f(\cdot)$ is a monotonically increasing function;
  - $g_i(x)$ is defined as a **discriminant function**.
- To classify input $x$ as the $i$-th class, we adopt the decision test:
  - $p(y_i|x) > p\big(y_j\big|x\big)$. This is a general function for $\forall j \neq i$.
  - With discriminant function, the above decision test is restated: classify as $i$-th class if $g_i(x) > g_j(x), \forall j \neq i$.
  - Decision hypersurfaces are described as $g_{ij}(x) \equiv g_i(x) - g_j(x) = 0$.

© Copyright National University of Singapore. All Rights Reserved.

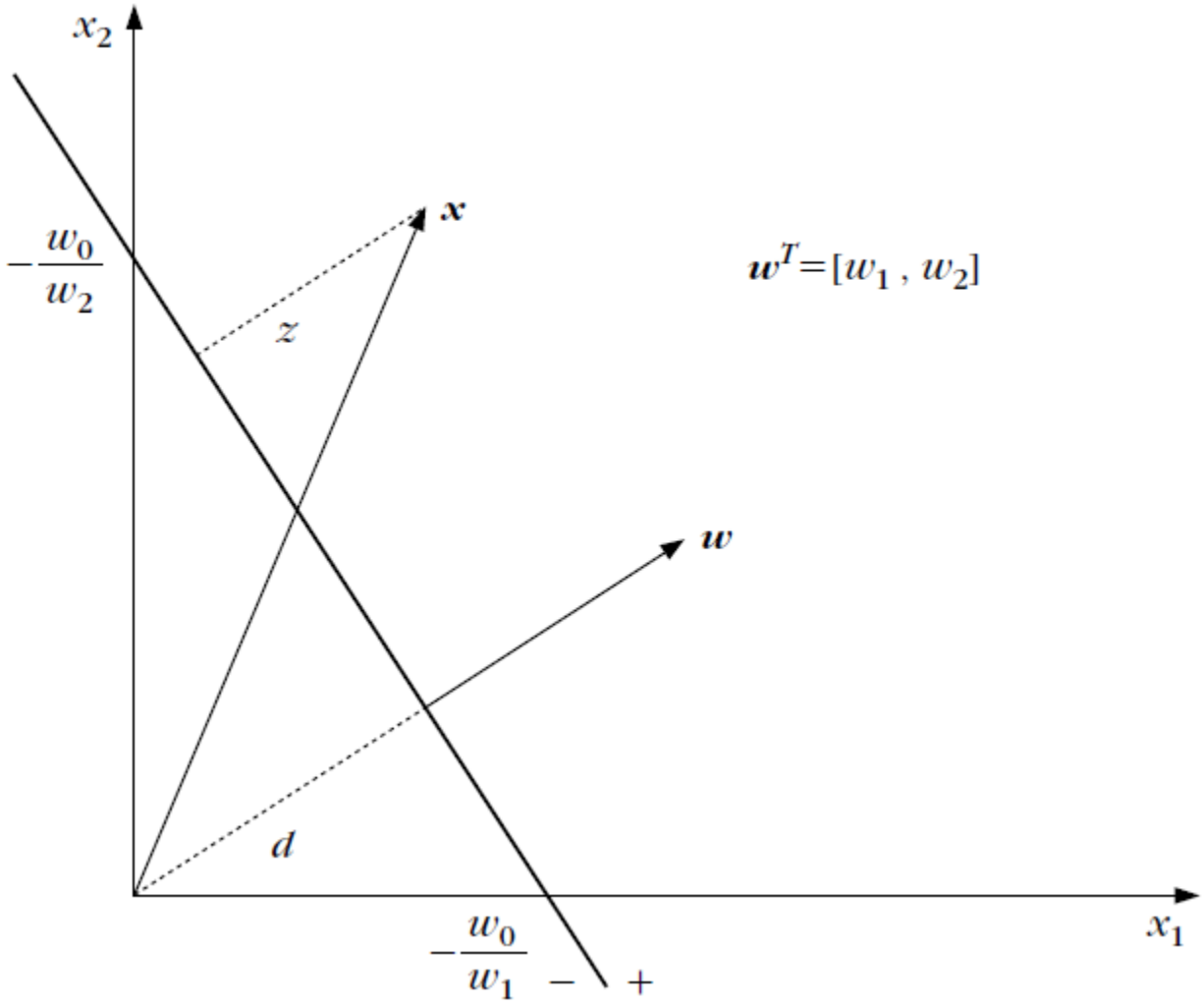
# Linear Discriminant Functions

- For a binary classification task, we may consider leveraging a linear discriminant function:
  - The decision hypersurface in the feature space is thus a hyperplane:
  - $g(x) = \boldsymbol{w}^T x + w_0$, $\boldsymbol{w}$ – the weight vector; $w_0$ – the threshold.
  - If the hypersurface is in a $l$-dim feature space, then $\boldsymbol{w} = [w_1, w_2, \dots, w_l]^T$.
- Suppose $x_1$ and $x_2$ are two points on the decision hyperplane:
  - $0 = \boldsymbol{w}^T x_1 + w_0 = \boldsymbol{w}^T x_2 + w_0 \Rightarrow \boldsymbol{w}^T (x_1 - x_2) = 0$
  - What can we infer from this equation?

© Copyright National University of Singapore. All Rights Reserved.

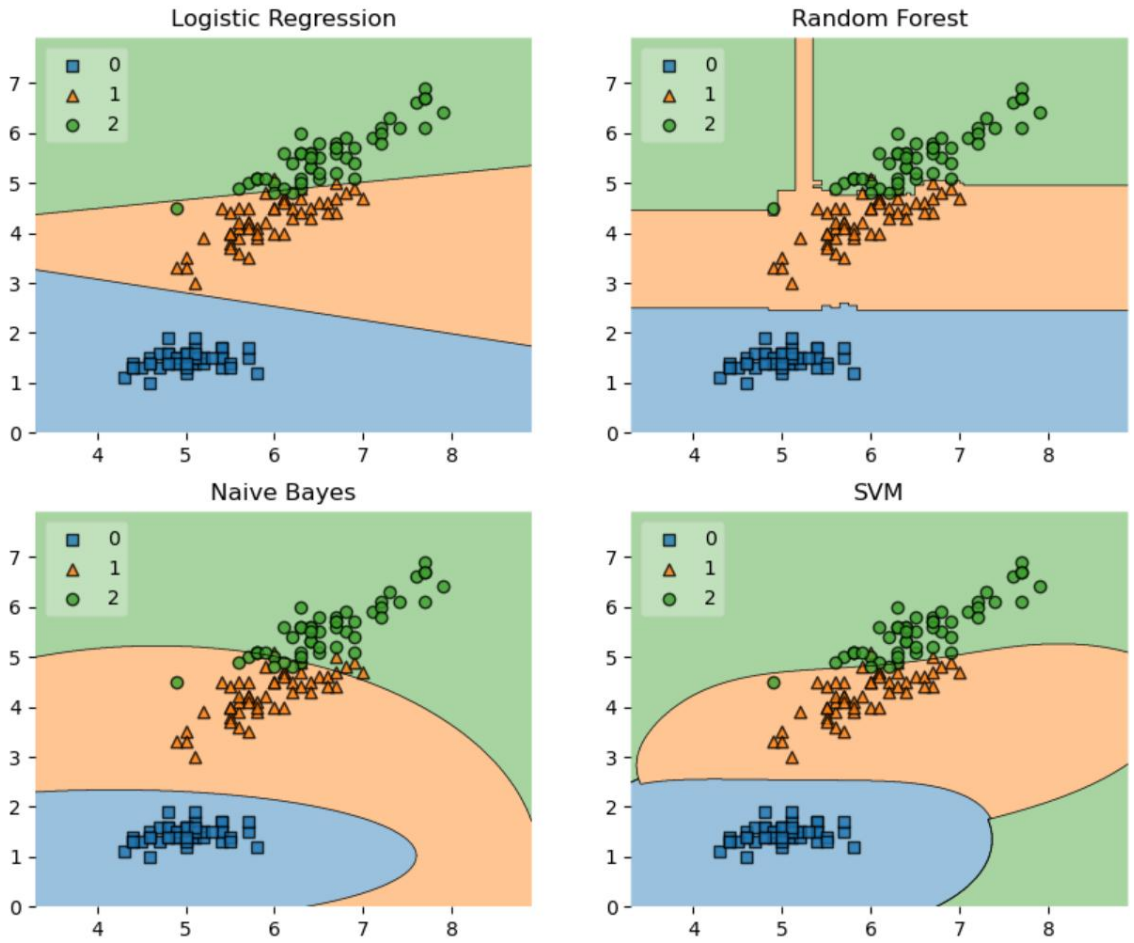
# Visualize the Decision Hyperplane

- $0 = \boldsymbol{w}^T x_1 + w_0 = \boldsymbol{w}^T x_2 + w_0 \Rightarrow \boldsymbol{w}^T(x_1 - x_2) = 0$
- What can we infer from this equation?
- Observe: the difference vector $x_1 - x_2$ lies on the decision hyperplane.
- Thus, weight vector $\boldsymbol{w}$ must be orthogonal to the decision hyperplane!




© Copyright National University of Singapore. All Rights Reserved.

# Decision Hyperplanes: Example




© Copyright National University of Singapore. All Rights Reserved.

# The Perceptron Algorithm: Basics

- Given $g(x) = \boldsymbol{w}^T x + w_0$, our concern: compute the unknown parameters $w_i$, $i = 0,1,\ldots,D$.
  - First assume that the two classes $y_1$ and $y_2$ are **linearly separable**.
  - Mathematically, assume that there exists a hyperplane defined by $\boldsymbol{w}^{*T} x = 0$ such that $\boldsymbol{w}^{*T} x > 0, \forall x \in y_1$ and $\boldsymbol{w}^{*T} x < 0, \forall x \in y_2$.
  - Note that the formulation covers the case where the hyperplane does not cross the origin (as in the last slide): $\boldsymbol{w}^{*T} x + w_0^* = 0$.
  - Under such case, we can further define a $(D+1)$-dim feature vector $x' \equiv [x^T, 1]^T$ and $\boldsymbol{w}' \equiv [\boldsymbol{w}^{*T}, w_0^*]^T$, thus $\boldsymbol{w}^{*T} x + w_0^* = \boldsymbol{w}'^T x'$.
- Back to the question: how to compute the unknown parameters?

© Copyright National University of Singapore. All Rights Reserved.

# The Perceptron Algorithm: Optimization

- Compute the unknown parameters $w_i$, $i = 0,1,\ldots,D$ for $\boldsymbol{w}^{*T}x = 0$.

- View the problem as a typical **optimization** task:
  - An appropriate cost function;
  - An algorithmic optimization scheme to optimize the cost function.
  - Any simple but still relatively accurate cost function?

  - The perceptron cost: $J(\boldsymbol{w}) = \sum_{x \in M}(\delta_x \boldsymbol{w}^T x)$, where $M$ is the subset of mis-classified training vectors. Set $\delta_x = -1, \forall x \in y_1$ and $\delta_x = 1, \forall x \in y_2$.
  - When the cost function takes its minimum value, which is 0, can safely assume that a solution has been obtained, since all training feature vectors are correctly classified.

© Copyright National University of Singapore. All Rights Reserved.

# The Perceptron Algorithm: Optimization

- The perceptron cost: $J(\boldsymbol{w}) = \sum_{x \in M}(\delta_x \boldsymbol{w}^T x)$. How to optimize?
  - The perceptron cost function is continuous and piecewise linear.
  - If $\boldsymbol{w}$ changes smoothly, the cost $J(\boldsymbol{w})$ changes linearly until the point at which there is a change in the number of misclassified vectors.
  - At these points the gradient is not defined, and the gradient function is discontinuous.
  - Next: how to optimize the perceptron cost (to obtain the min value)?
  - Iterative minimization in the spirit of **gradient descent**. Defined at all points where the gradient function is continuous:

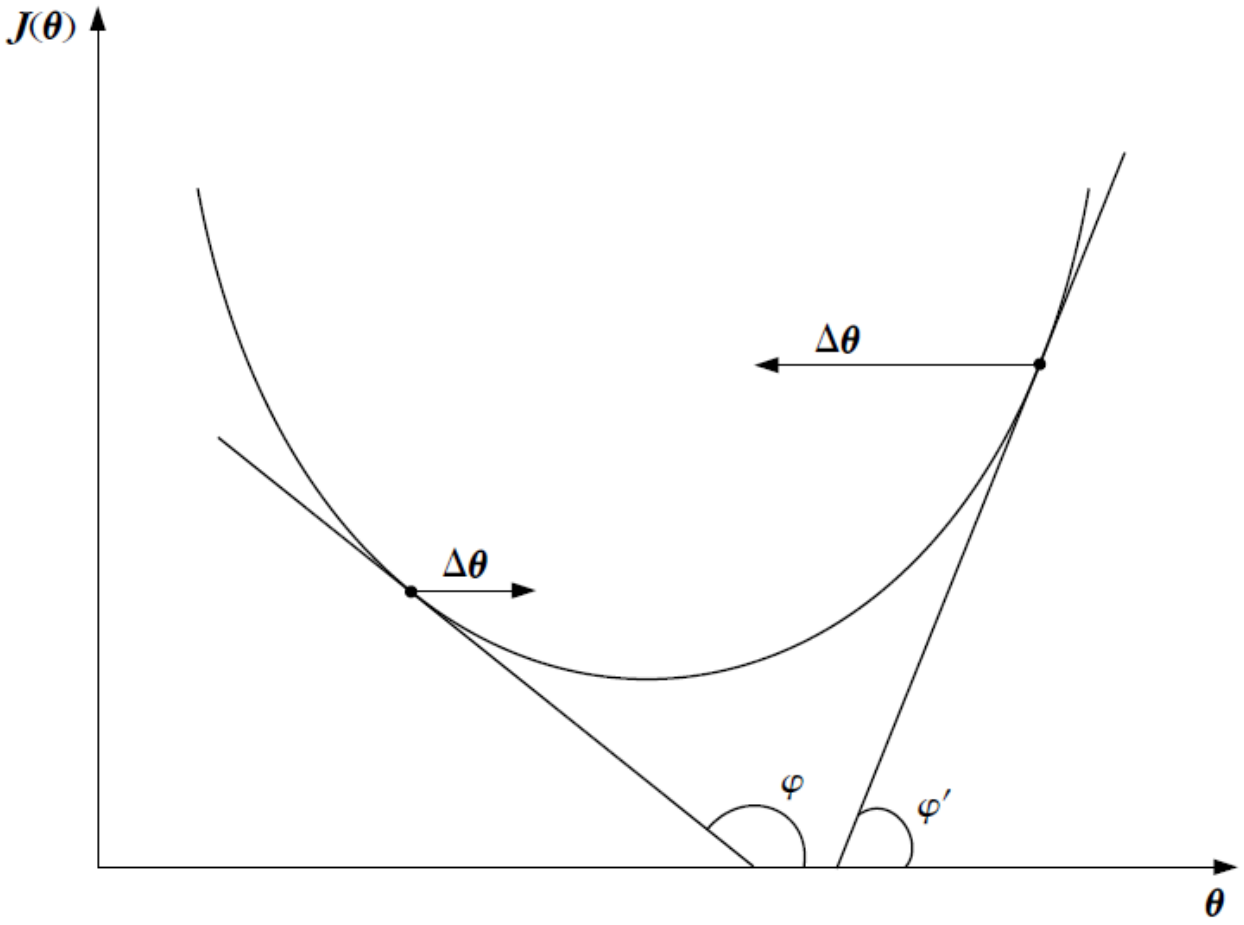
$$\boldsymbol{w}(t+1) = \boldsymbol{w}(t) - \rho_t \frac{\partial J(\boldsymbol{w})}{\partial \boldsymbol{w}}\Big|_{\boldsymbol{w}=\boldsymbol{w}(t)}$$

© Copyright National University of Singapore. All Rights Reserved.

# Optimization with Gradient Descent

$$\boldsymbol{w}(t+1) = \boldsymbol{w}(t) - \rho_t \frac{\partial J(\boldsymbol{w})}{\partial \boldsymbol{w}}\Big|_{\boldsymbol{w}=\boldsymbol{w}(t)}$$

- In the gradient descent scheme, the correction of the parameters takes place in the direction that decreases the value of the cost function.
- In reality, most cost functions are much more complicated: there exists local and global minimum, as well as saddle points.




© Copyright National University of Singapore. All Rights Reserved.

# The Perceptron Algorithm:

$$\boldsymbol{w}(t+1) = \boldsymbol{w}(t) - \rho_t \frac{\partial J(\boldsymbol{w})}{\partial \boldsymbol{w}}\Big|_{\boldsymbol{w}=\boldsymbol{w}(t)}, \text{where } J(w) = \sum_{x \in M} (\delta_x \boldsymbol{w}^T x)$$

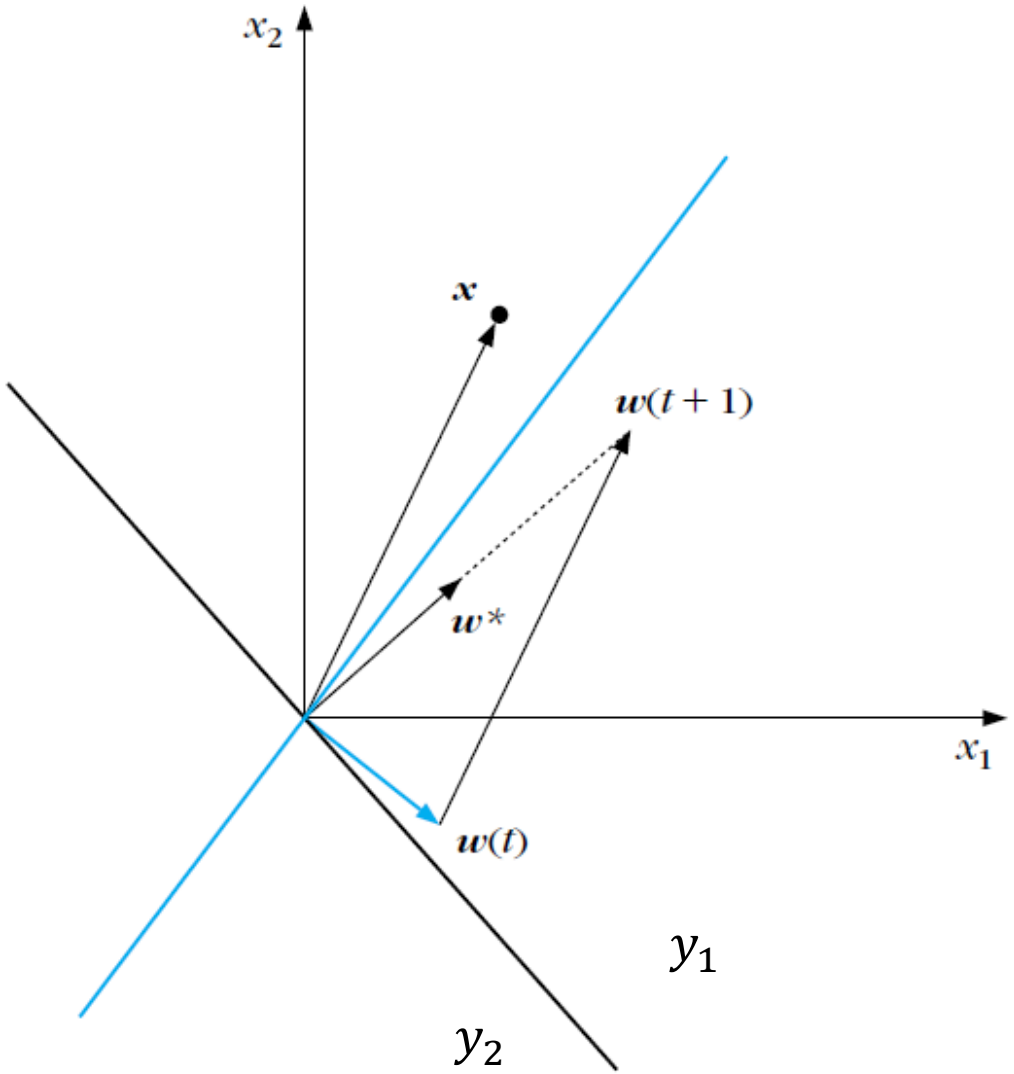
- At the points where the gradient is valid, we have $\frac{\partial J(w)}{\partial w} = \sum_{x \in M} \delta_x x$. The optimization function is thus: $\boldsymbol{w}(t+1) = \boldsymbol{w}(t) - \rho_t \sum_{x \in M} \delta_x x$.
- This is the perceptron algorithm, with an arbitrary starting point $\boldsymbol{w}(0)$.

- Choose $\boldsymbol{w}(0)$ randomly
- Choose $\rho_0$
- $t = 0$
- Repeat
  - $M = \emptyset$
  - For $i = 1$ to $N$
    - If $\delta_{x_i} \boldsymbol{w}(t)^T \boldsymbol{x}_i \geq 0$ then $M = M \cup \{\boldsymbol{x}_i\}$
  - End {For}
  - $\boldsymbol{w}(t+1) = \boldsymbol{w}(t) - \rho_t \sum_{x \in M} \delta_x x$.
  - Adjust $\rho_t$
  - $t = t + 1$
- Until $M = \emptyset$

© Copyright National University of Singapore. All Rights Reserved.

# Visualizing the Perceptron Algorithm

- Geometric interpretation of the perceptron algorithm:
- The update of the weight vector is in the direction of $x$ in order to turn the decision hyperplane to include $x$ in the correct class



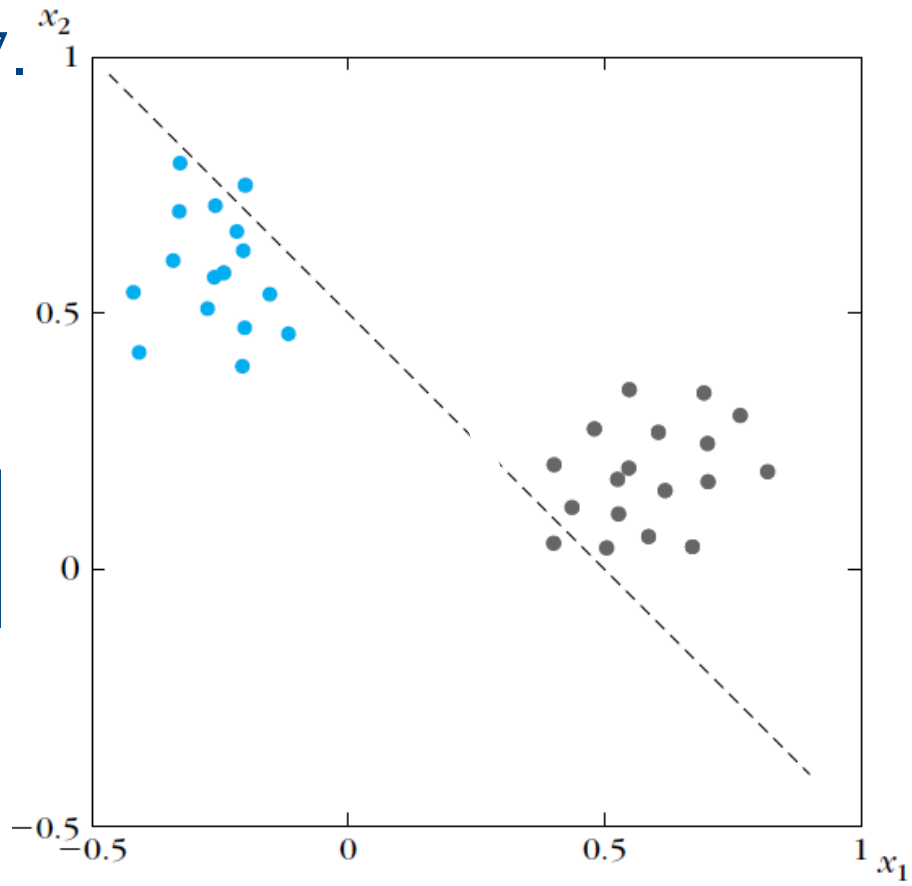

© Copyright National University of Singapore. All Rights Reserved.

# The Perceptron Algorithm: an Example

- Dashed line: initial hyperplane $x_1 + x_2 - 0.5 = 0$.
- Weight vector: $\boldsymbol{w}^T = [1,1,-0.5]^T$; set $\rho_t = \rho = 0.7$.
- What can we observe?
- The line classifies correctly all the vectors (pts) except 2: $[0.4, 0.05]^T$ and $[-0.20, 0.75]^T$.
- The next weight vector is updated as:

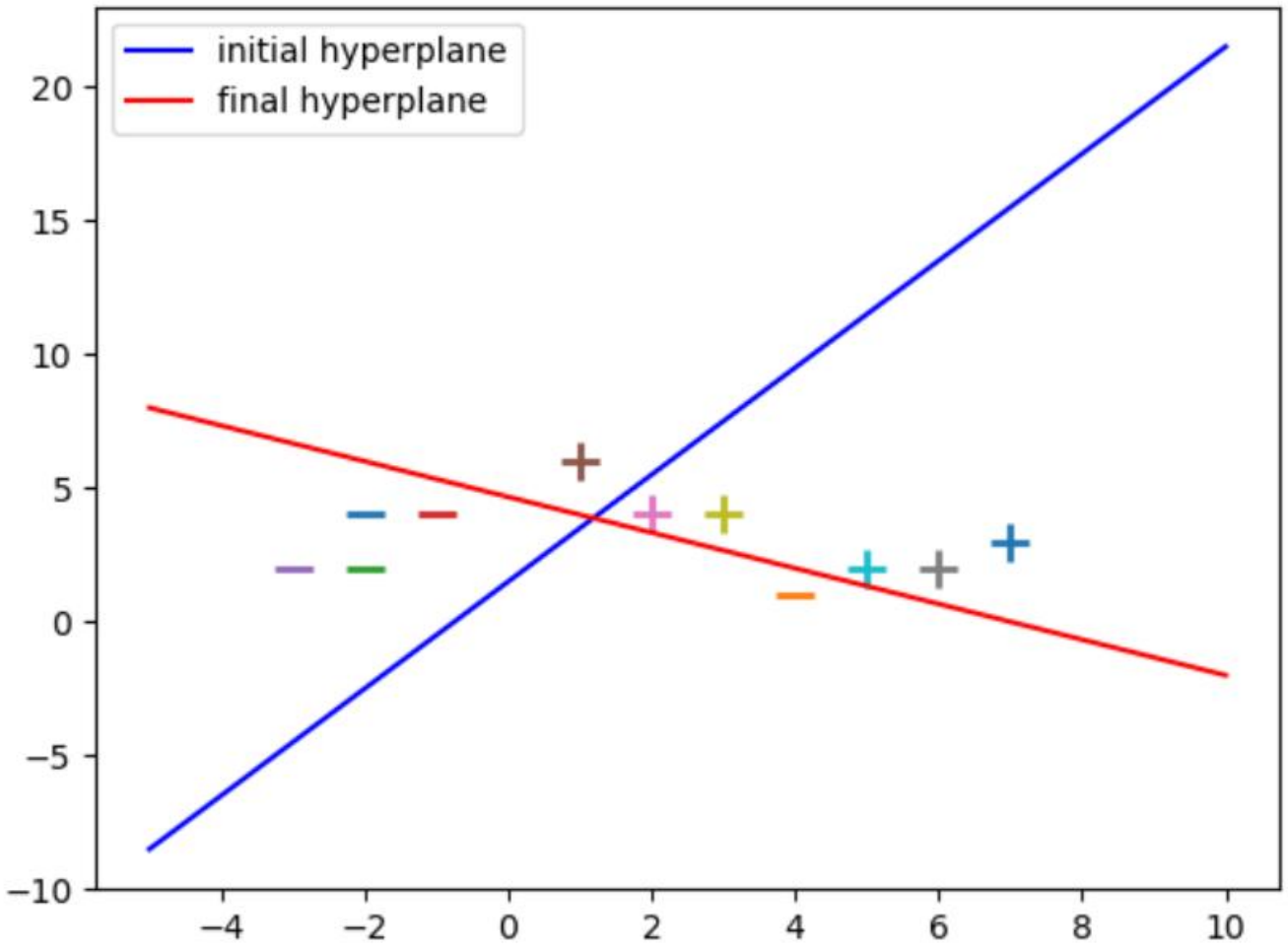
$$\boldsymbol{w}(t+1) = \begin{bmatrix} 1 \\ 1 \\ -0.5 \end{bmatrix} - 0.7(-1)\begin{bmatrix} 0.4 \\ 0.05 \\ 1 \end{bmatrix} - 0.7(+1)\begin{bmatrix} -0.2 \\ 0.75 \\ 1 \end{bmatrix} = \begin{bmatrix} 1.42 \\ 0.51 \\ -0.5 \end{bmatrix}$$

- This corresponds to the new (solid) line:
- $1.42x_1 + 0.51x_2 - 0.5 = 0$, which classifies all points correctly, algorithm is thus terminated.




© Copyright National University of Singapore. All Rights Reserved.

# The Perceptron Algorithm: Implementation




© Copyright National University of Singapore. All Rights Reserved.

# Variants of the Perceptron Algorithm

- A simpler form of the perceptron algorithm – cyclical form:
  - The training data enter the algorithm **cyclically**, **one by one**.
  - If the algorithm has not converged after the presentation of all the samples once, then the procedure keeps repeating until convergence, i.e, when all training samples have been classified correctly.
  - Let $w(t)$ be the weight vector estimated and $x_t$ the corresponding data presented at the $t$-th iteration step, the algorithm is stated as:
  - $w(t+1) = w(t) + \rho x_t, \text{if } x_t \in y_1 \text{ and } w^T(t)x_t \leq 0$ (note similarity);
  - $w(t+1) = w(t) - \rho x_t, \text{if } x_t \in y_2 \text{ and } w^T(t)x_t \geq 0;$
  - $w(t+1) = w(t), \text{otherwise}.$
  - How to interpret this variant of the perceptron algorithm?

© Copyright National University of Singapore. All Rights Reserved.

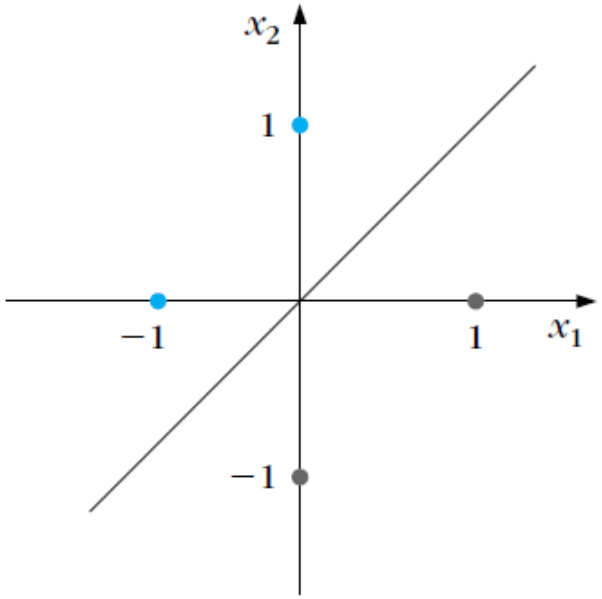
# Variants of the Perceptron Algorithm

- $w(t+1) = w(t) + \rho x_t, \text{if } x_t \in y_1 \text{ and } w^T(t)x_t \leq 0;$
- $w(t+1) = w(t) - \rho x_t, \text{if } x_t \in y_2 \text{ and } w^T(t)x_t \geq 0;$
- $w(t+1) = w(t), \text{otherwise}.$

- If the current training sample is classified correctly, no action is taken. If the sample is misclassified, the weight vector is corrected by adding/subtracting an amount proportional to $x_t$.
- The algorithm belongs to a more general algorithmic family: the **reward and punishment schemes**.
  - If the classification is correct, the **reward** is that no action is taken. Otherwise, the **punishment** is the cost of correction.

© Copyright National University of Singapore. All Rights Reserved.

# Variants of the Perceptron Algorithm: Example




- 4 points are as shown in the feature space: blue being class 1 $y_1$ and black being class 2 $y_2$.
  - How to design a linear classifier using the perceptron algorithm in its **reward and punishment form**?
  - Set $\rho = 1$ and initial weight vector $\boldsymbol{w}(0) = [0, 0, 0]^T$

Ste Step 7.

$$\boldsymbol{w}^T(6)\begin{bmatrix} 0 \\ -1 \\ 1 \end{bmatrix} = -1 < 0, \quad \boldsymbol{w}(7) = \boldsymbol{w}(6) = \begin{bmatrix} -1 \\ 1 \\ 0 \end{bmatrix}$$

- Since for four consecutive steps no correction is needed, all points are correctly classified and the algorithm **terminates**.

Step 6.

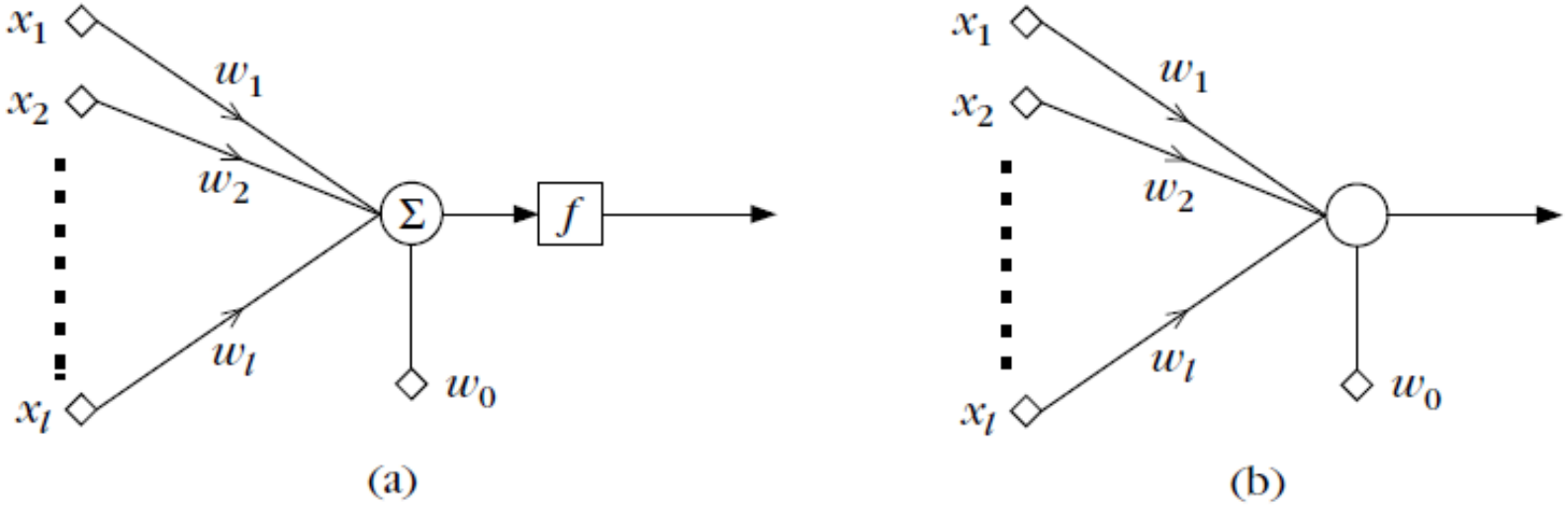
$$\boldsymbol{w}^T(5)\begin{bmatrix} 0 \\ 1 \\ 1 \end{bmatrix} = 1 > 0, \quad \boldsymbol{w}(6) = \boldsymbol{w}(5)$$

- The solution is $\boldsymbol{w} = [-1, 1, 0]^T$, which corresponds to the linear classifier $-x_1 + x_2 = 0$.

© Copyright National University of Singapore. All Rights Reserved.

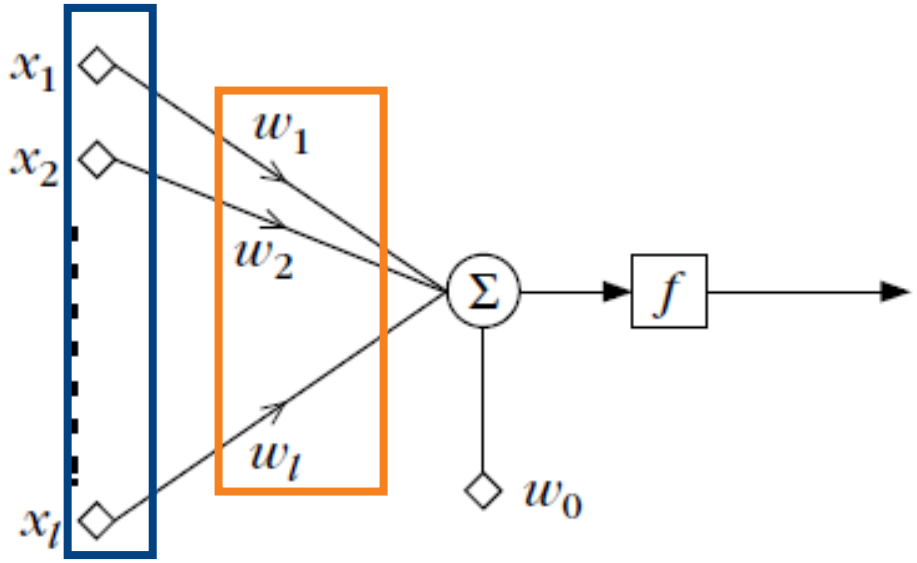
# The Perceptron

- The goal of the perceptron algorithm: obtain a linear classifier that is applicable to new (and unknown) data:
  - I.e., classify the new data into the two classes (binary classification).
  - IF $w^T x + w_0 > 0$, assign $x$ to $y_1$;
  - ELSE IF $w^T x + w_0 < 0$, assign $x$ to $y_2$;




© Copyright National University of Singapore. All Rights Reserved.

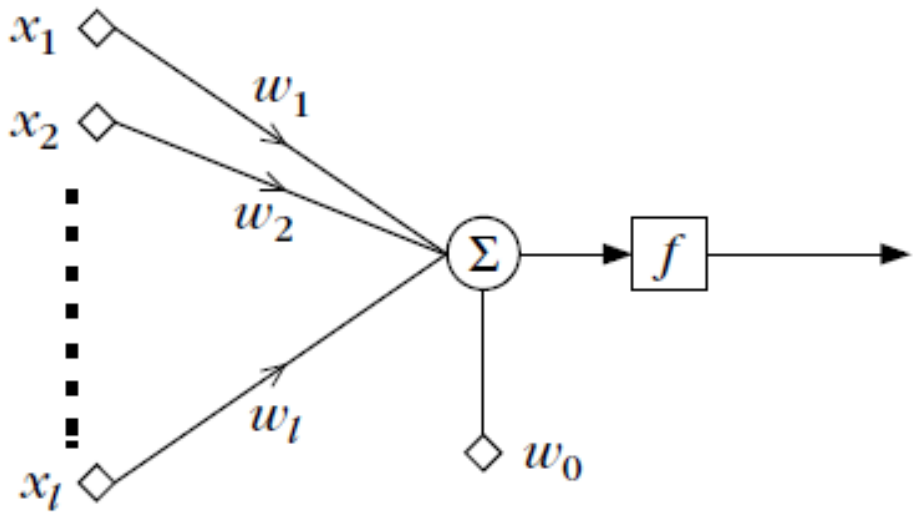
# The Perceptron: Details




- Let's describe the perceptron in detail:
- The elements of the feature vector $x$ are applied to the *input nodes* (blue).
- Then each one is multiplied by the corresponding weights known as *synaptic weights* or simply *synapses* (orange).
- The products are summed up with the threshold $w_0$. The result then goes through a nonlinear device, which implements the *activation function* $f$.
- Wide range of choices for $f$, example: the ***hard limiter*** step function $f(z) = -1 \text{ if } z < 0$ and $f(z) = 1 \text{ if } z > 0$.

© Copyright National University of Singapore. All Rights Reserved.

# The Perceptron




- This basic network is known as a perceptron or neuron.
- Perceptrons are simple examples of the **learning machines** – i.e., structures whose free parameters are updated by a **learning algorithm**, such as the perceptron algorithm, to “learn” a specific task, based on a set of training data.
- The perceptron can be used as the basic building element for more complex learning networks.
- Sometimes a neuron with a hard limiter device is referred to as a **McCulloch-Pitts neuron**.

© Copyright National University of Singapore. All Rights Reserved.

# Extending to Multi-Class

- Previously, we default our task as a binary classification task – how can we generalize to multi-class ($M$-class) scenarios?
- Generalize in the simplest manner:
  - A linear discriminant function (defined by a weight vector $\boldsymbol{w}_i$) is defined for each of the $M$ classes.
  - Input feature (data) $x$ is classified as the $i$-th class if the following holds:

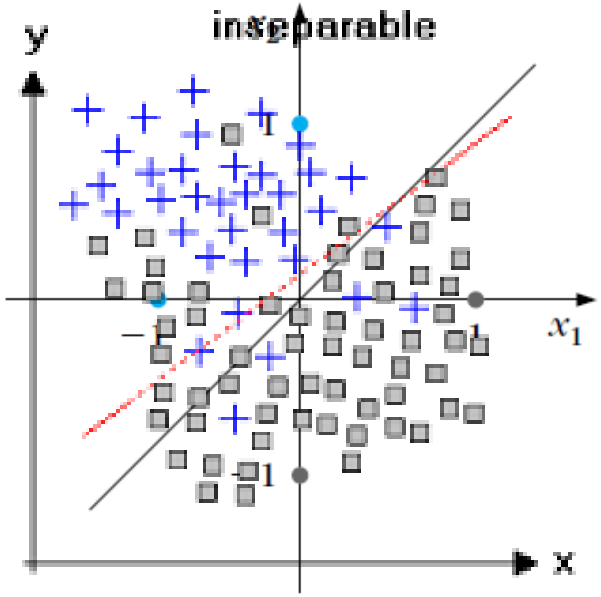
$$\boldsymbol{w}_i^T x > \boldsymbol{w}_j^T x, \forall j \neq i$$

  - Discussion: how can we construct a linear classifier that could perform this comparison across all classes in one-shot?

© Copyright National University of Singapore. All Rights Reserved.

# Linearly Separable?

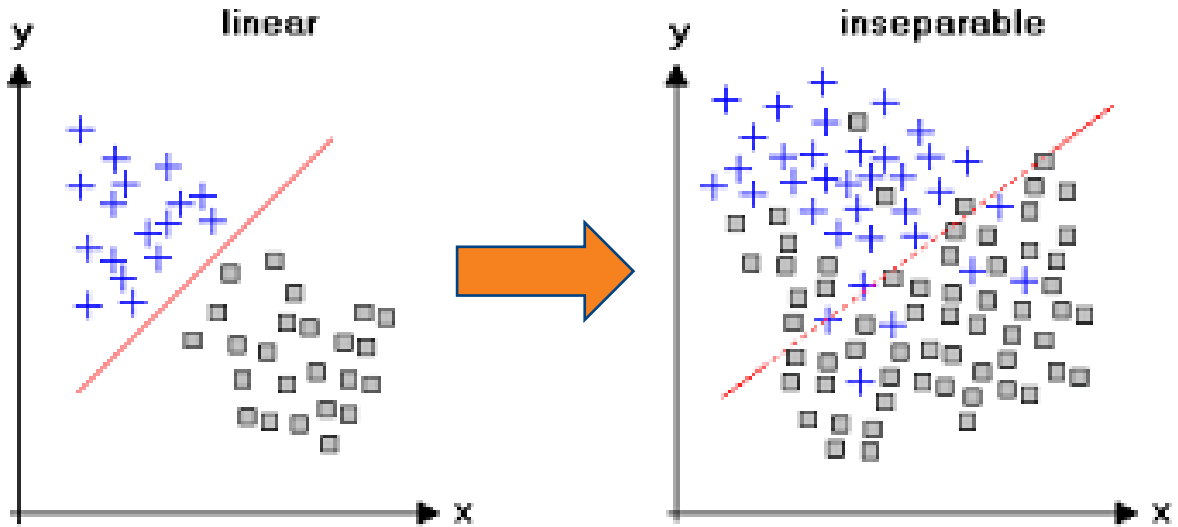
- From previous slides, we could observe how simple the formulation of a linear classifier is.
- Our discussion: all data can be classified as one of the classes;
  - I.e., a strong assumption that the classes are linearly separable.
- In many cases, the class are not linearly separable – but still want to leverage the simplicity of linear classifiers.
  - Suboptimal, but simple → best of all suboptimals;
  - Compute the weight vectors under a suitable optimality criterion;
  - A simple criterion to start with – mean square error.




© Copyright National University of Singapore. All Rights Reserved.

# Classification with (Mean Square) Error

- Our goal: design a linear classifier so that the desired outputs of the data are $\pm 1$ – but with errors.
  - The true output will not always be equal to the desired output.
  - Given input $x$, the actual output is $\boldsymbol{w}^T x$, the desired output is $y(x)$.
  - $\boldsymbol{w}$ is computed to minimize the mean square error (MSE) between the actual output and the desired output.




© Copyright National University of Singapore. All Rights Reserved.

# MSE Estimation

- Formulation of the MSE between the desired and actual output:

$$J(\boldsymbol{w}) = \mathbb{E}[|y(x) - x^T\boldsymbol{w}|^2], \qquad \widehat{\boldsymbol{w}} = \arg\min_{w} J(\boldsymbol{w})$$

- Minimizing the above equation w.r.t. $w$ as:

$$\frac{\partial J(\boldsymbol{w})}{\partial \boldsymbol{w}} = 2\mathbb{E}[x(y - x^T\boldsymbol{w})] = 0 \Rightarrow \widehat{\boldsymbol{w}} = R_x^{-1}\mathbb{E}[xy]$$

  - Here, $R_x = \mathbb{E}[xx^T]$ is the correlation, which is equal to the covariance matrix, $\mathbb{E}[xy]$ is the cross-correlation between the input and the desired output, formulated as:

$$\mathbb{E}[xy] = \mathbb{E}\left[\begin{bmatrix} x_1 y \\ \vdots \\ x_l y \end{bmatrix}\right]$$

© Copyright National University of Singapore. All Rights Reserved.

# Sum of Error Squares Estimation

- A criterion closely related to the MSE is the sum of error squares or simply the *least squares error* (LSE) criterion:

$$J(\boldsymbol{w}) = \sum_{i}^{N} \left(y(x_i) - x_i^T \boldsymbol{w}\right)^2 = \sum_{i}^{N} \left(y_i - x_i^T \boldsymbol{w}\right)^2 \equiv \sum_{i}^{N} e_i^2$$

- Compare with MSE?
  - Errors btw the desired output and the true output are ***summed*** over all the training feature vectors, instead of **averaging** out.
  - Overcome the need for explicit knowledge of the underlying probability distribution functions (pdfs).

© Copyright National University of Singapore. All Rights Reserved.

# LSE Estimation

- Minimizing the LSE formulation $J(\boldsymbol{w}) = \sum_i^N \left(y_i - x_i^T \boldsymbol{w}\right)^2$ w.r.t. $w$ results in:

$$\sum_i^N x_i\left(y_i - x_i^T \widehat{\boldsymbol{w}}\right) = 0 \Rightarrow \left(\sum_i^N x_i x_i^T\right)\widehat{\boldsymbol{w}} = \sum_i^N x_i y_i$$

- Mathematically, we can define two matrices:

$$X = \begin{bmatrix} x_1^T \\ x_2^T \\ \vdots \\ x_N^T \end{bmatrix} = \begin{bmatrix} x_{11} & x_{12} & \cdots & x_{1D} \\ x_{21} & x_{22} & \cdots & x_{2D} \\ \vdots & \vdots & \ddots & \vdots \\ x_{N1} & x_{N2} & \cdots & x_{ND} \end{bmatrix}, \qquad Y = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_N \end{bmatrix}$$

- I.e., $X$ is an $N \times D$ matrix whose rows are the available training feature vectors, and $Y$ is a vector consisting of the corresponding desired outputs.

© Copyright National University of Singapore. All Rights Reserved.

# LSE Estimation

- With the matrices, we have $\sum_i^N x_i x_i^T = X^T X$ and $\sum_i^N x_i y_i = X^T Y$. Hence the equation for optimal weight vector is formulated as:
- $(X^T X)\widehat{w} = X^T Y \Rightarrow \widehat{w} = (X^T X)^{-1} X^T Y$
  - Here we define $X^T X$ as the sample correlation matrix, $X^+ \equiv (X^T X)^{-1} X^T$ is usually known as the *pseudoinverse* of matrix $X$, which is only valid if the matrix $(X^T X)$ is invertible.
  - In the case where $N = D$, the optimal weight vector satisfies the equation of: $X\widehat{w} = Y$.
  - In most cases where $N > D$, the solution obtained from $\widehat{w} = X^T Y$ is the vector that minimizes the sum of squared errors.
  - Note that the LSE solution tends to the MSE solution when $N$ is large!

© Copyright National University of Singapore. All Rights Reserved.

# Over- and Under-Determined Systems

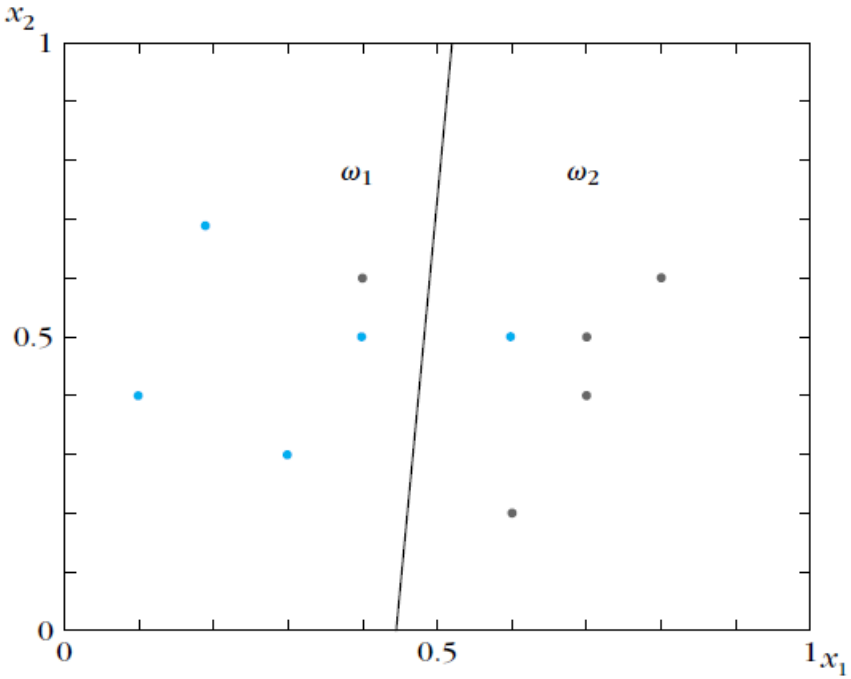
$$(X^T X)\hat{w} = X^T Y \Rightarrow \hat{w} = (X^T X)^{-1} X^T Y$$

- During the computation of the above equation, we assume $N > l$, i.e., we have more samples than the number of features per sample.
- We have more than enough equations to determine the linear classifier – denoted as an over-determined system.
- Question: what if $N < D$, i.e., we do NOT have enough samples?
  - There will be more than one unique solution!
  - We then need to constrain the search. Assume $(XX^T)$ is invertible, denote $w = X^T a$, then we have $XX^T a = y$ from $Xw = y$.
  - Thus $\hat{a} = (XX^T)^{-1} y$, thus $\hat{w} = X^T (XX^T)^{-1} y$
  - Here we denote $X^\dagger = X^T (XX^T)^{-1}$ as the right inverse of $X$.
- Note: our discussion will still focus on over-determined systems!

© Copyright National University of Singapore. All Rights Reserved.

# LSE Estimation: an Example

- Class $y_1$ consists of the vectors $[0.2, 0.7]^T, [0.3, 0.3]^T, [0.4, 0.5]^T, [0.6, 0.5]^T, [0.1, 0.4]^T$ and $y_2$ of $[0.4, 0.6]^T, [0.6, 0.2]^T, [0.7, 0.4]^T, [0.8, 0.6]^T, [0.7, 0.5]^T$.
  - Goal: a linear classifier of the form $w_1 x_1 + w_2 x_2 + w_0 \times 1 = 0$.
  - With LSE estimation: extend each vector with 1 as the 3rd dimension;
  - Then compute/form matrices $X$ and $Y$, note that $Y$ consists of five 1's and five -1's, and compute $X^T X$ and $X^T Y$ as:
  - $X^T X = \begin{bmatrix} 2.8 & 2.24 & 4.8 \\ 2.24 & 2.41 & 4.7 \\ 4.8 & 4.7 & 10 \end{bmatrix}; X^T Y = \begin{bmatrix} -1.6 \\ 0.1 \\ 0.0 \end{bmatrix}$
  - The optimal $w$ is thus computed as:
  - $\hat{w} = (X^T X)^{-1} X^T Y = \begin{bmatrix} -3.218 \\ 0.241 \\ 1.431 \end{bmatrix}$




© Copyright National University of Singapore. All Rights Reserved.

# The Capabilities of Linear Classifiers

- As discuss, the linear classifiers $g(x) = \boldsymbol{w}^T x + w_0$ can be viewed as learning machines which attempt to estimate the corresponding class label $y$.
- In practice, (and discussed for LSE estimation), the functions $g(\cdot)$ are estimated using a finite training dataset denoted as $\mathcal{D} = \{(y_i, x_i), i = 1,2, \dots, N\}$ and a suited method (MSE/LSE).
  - To emphasize the dependency of $g$ over $\mathcal{D}$, we write $g(x; \mathcal{D})$.
- The ultimate goal of $g$ is to be applicable to new data – the capabilities of $g$ to approximate the real class and how this is affected by the finite training set?

© Copyright National University of Singapore. All Rights Reserved.

# The Capabilities of Linear Classifiers

- Note that the approximation of the real class may be **very good** for a specific training data set but **very bad** for another.
- The effectiveness of $g(x;\mathcal{D})$ can be evaluated by computing its mean square deviation from the desired optimal value (essentially, the MSE):

$$\mathbb{E}_{\mathcal{D}}\left[\left(g(x;\mathcal{D})-y(x)\right)^2\right]$$

- The above formulation can be further re-formulated as:

$$\mathbb{E}_{\mathcal{D}}\left[\left(g(x;\mathcal{D})-y(x)\right)^2\right]$$
$$=\left(\mathbb{E}_{\mathcal{D}}[g(x;\mathcal{D})]-y(x)\right)^2+\mathbb{E}_{\mathcal{D}}[(g(x;\mathcal{D})-\mathbb{E}_{\mathcal{D}}[g(x;\mathcal{D})])^2]$$

- The first term is the contribution of the **bias** and the second is the contribution of the **variance**.

© Copyright National University of Singapore. All Rights Reserved.

# End of Week 4

© Copyright National University of Singapore. All Rights Reserved.

# The Bias-Variance Dilemma: Bias

- Bias is formulated as: $\left(\mathbb{E}_{\mathcal{D}}[g(x;\mathcal{D})] - y(x)\right)^2$.
- Bias refers to the degree to which a model is able to accurately capture the underlying patterns in a dataset.
  - Layman term: the diversion from original answer is known as Bias.
  - We say $g(\cdot)$ is unbiased when $\mathbb{E}_{\mathcal{D}}[g(x;\mathcal{D})] = y(x)$.
  - A model with **high bias** is said to be **underfitting** the data, as it is not able to capture the complexity of the true underlying relationship.
  - How can we intuitively know if $g$ is of high or low bias?
  - High bias: poor performance on both **training data** and unseen data.

© Copyright National University of Singapore. All Rights Reserved.

# The Bias-Variance Dilemma: Variance

- Variance is formulated as: $\mathbb{E}_{\mathcal{D}}[(g(x;\mathcal{D}) - \mathbb{E}_{\mathcal{D}}[g(x;\mathcal{D})])^2]$.
- Variance refers to the degree to which a model is able to adapt to new data.
  - Layman term: the Variance measures the “spread” of a distribution.
  - A model with **high variance** is said to be **overfitting** the data, as it is too complex and has learned to **fit the noise** in the training data as well as the underlying patterns.
  - How can we intuitively know if $g$ is of high or low variance?
  - High variance: much poorer performance on unseen data compared to the training data. (Usually training performance is quite descent)

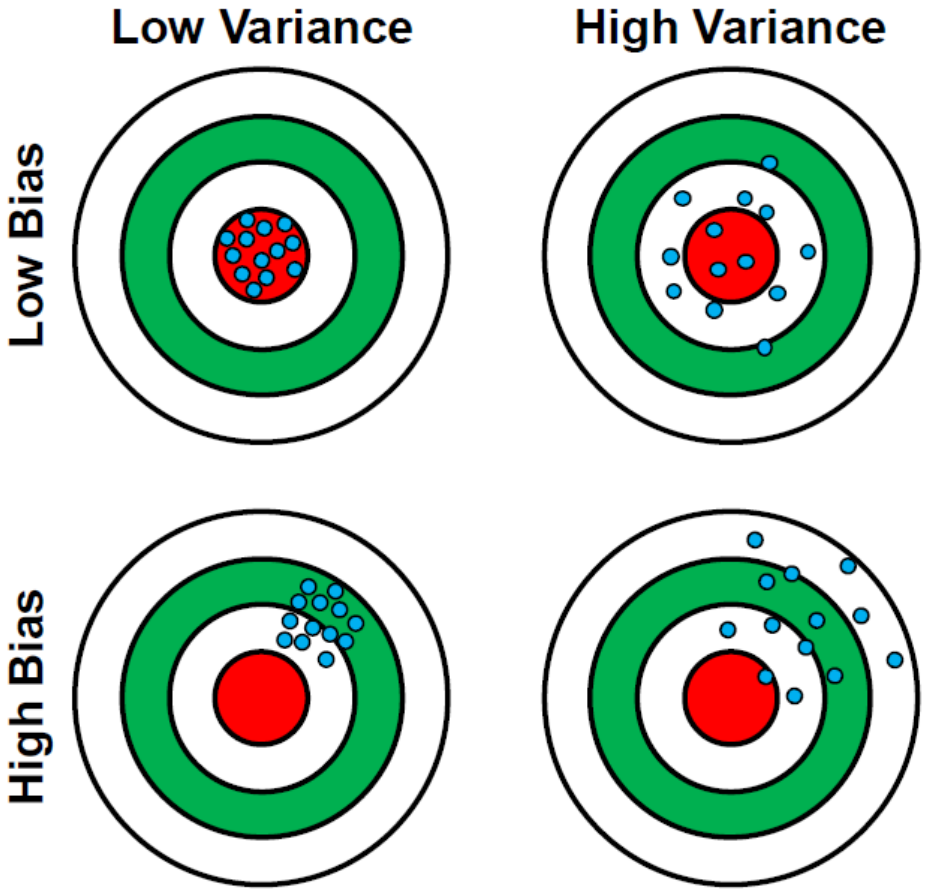
© Copyright National University of Singapore. All Rights Reserved.

# The Bias-Variance Dilemma

- Suppose we are trying to predict red target below.

**Low Bias**: blue predictions on average close to red target
**Low Variance**: low variability among blue predictions

**High Bias**: blue predictions on average not close to red target
**Low Variance**: Low variability among blue predictions




**Low Bias**: blue predictions on average close to red target
**High Variance**: large variability among blue predictions

**High Bias**: blue predictions on average not close to red target
**High Variance**: high variability among blue predictions

© Copyright National University of Singapore. All Rights Reserved.

# The Bias-Variance Dilemma

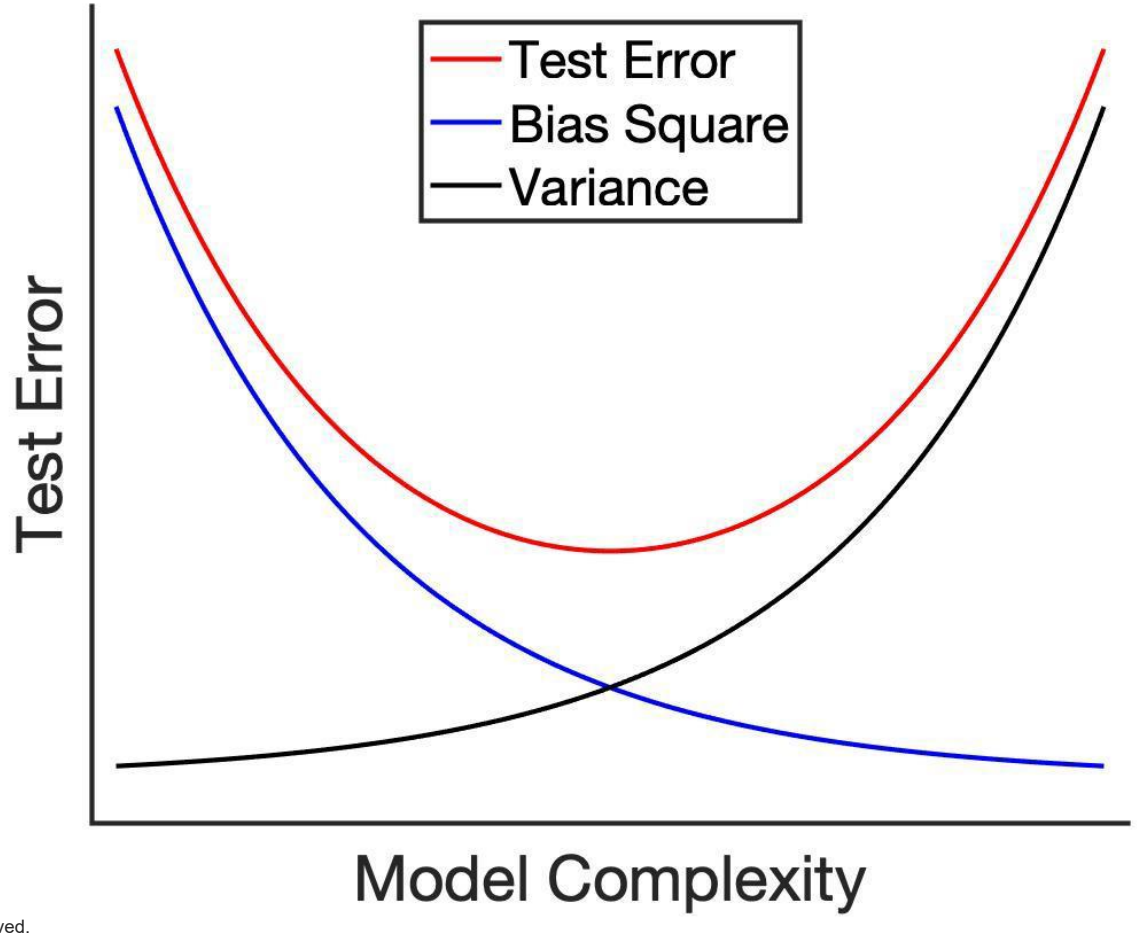
$$\mathbb{E}_{\mathcal{D}}\left[\left(g(x;\mathcal{D}) - y(x)\right)^2\right]$$
$$= \left(\mathbb{E}_{\mathcal{D}}[g(x;\mathcal{D})] - y(x)\right)^2 + \mathbb{E}_{\mathcal{D}}[(g(x;\mathcal{D}) - \mathbb{E}_{\mathcal{D}}[g(x;\mathcal{D})])^2]$$

- For a **finite data set**, there is a **trade-off** between these two terms.
- Increasing the bias decreases the variance and vice versa. This is known as the ***bias-variance dilemma***.
  - It is difficult to simultaneously minimize both bias and variance.
  - As the complexity of a model increases, its bias decreases, but its variance increases.
  - Conversely, as the complexity of a model decreases, its bias increases, but its variance decreases.
- The optimal level of complexity for a model depends on the **specific problem** and the **available data**.

© Copyright National University of Singapore. All Rights Reserved.

# The Bias-Variance Dilemma (Trade-off)

- Test error = Bias Squared + Variance + Irreducible Noise.




© Copyright National University of Singapore. All Rights Reserved.

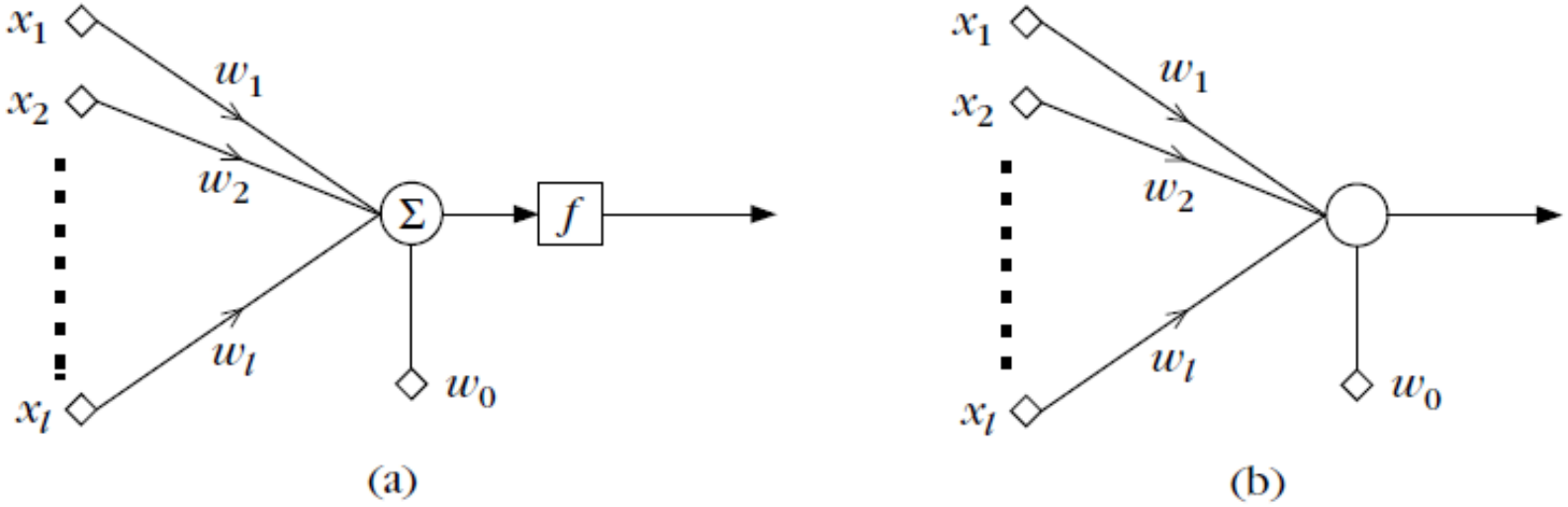
# Nonlinear Classifiers:

- ✓ Analysis of the XOR problem.
- ✓ The Multi-Layer Perceptron (MLP).
- ✓ Training the MLP with Backpropagation (BP).
- ✓ The Radial Basis Function Networks (RBF).

© Copyright National University of Singapore. All Rights Reserved.

# Recap: Linear Classifier – The Perceptron

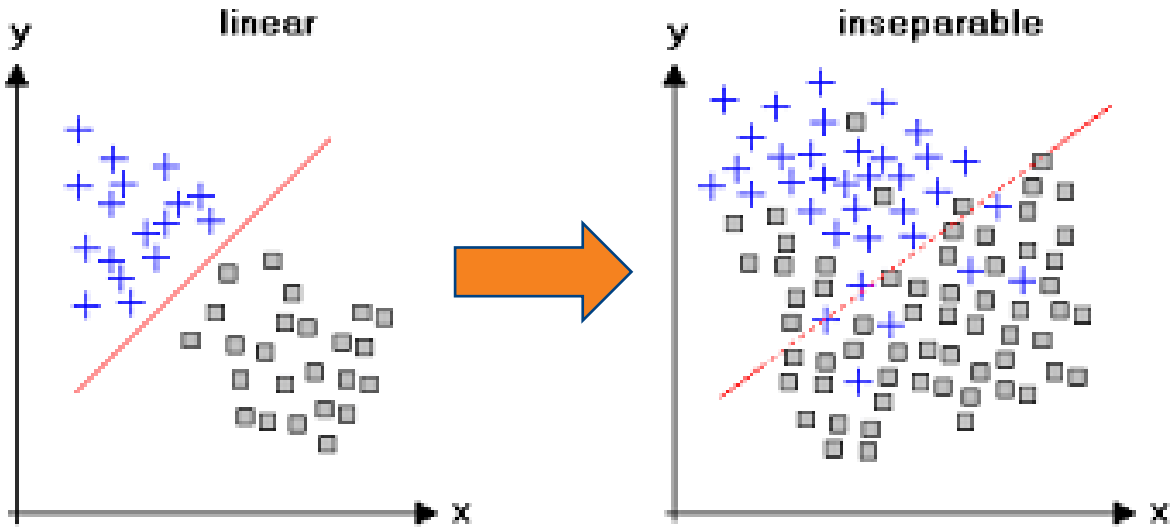
- The goal of the perceptron algorithm: obtain a linear classifier that is applicable to new (and unknown) data:
  - I.e., classify the new data into the two classes (binary classification).
  - IF $w^T x + w_0 > 0$, assign $x$ to $y_1$;
  - ELSE IF $w^T x + w_0 < 0$, assign $x$ to $y_2$;




© Copyright National University of Singapore. All Rights Reserved.

# Recap: Linear Classifier – Minimize Errors

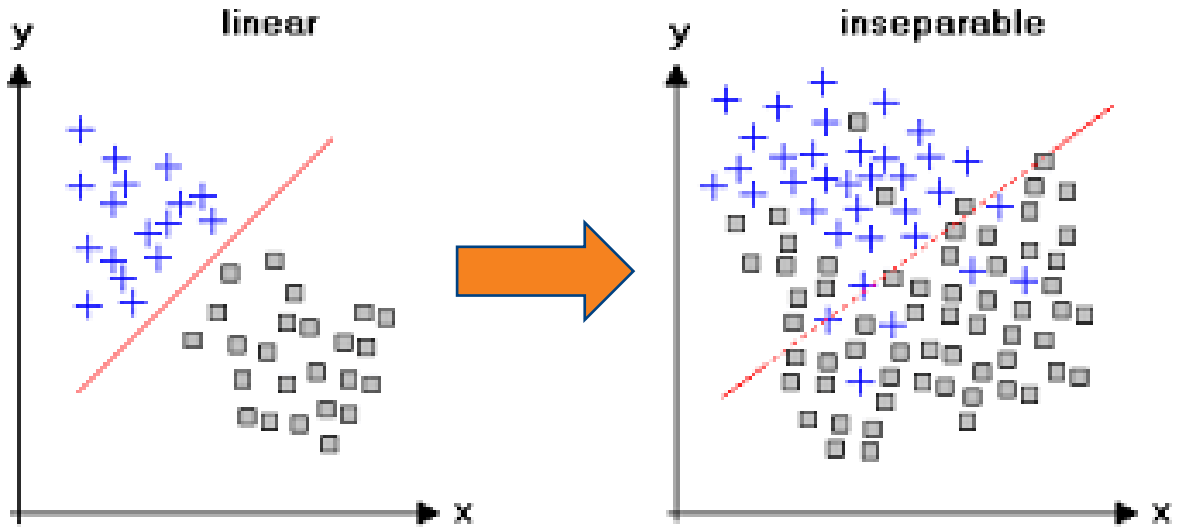
- Our goal: design a linear classifier so that the desired outputs of the data are $\pm 1$ – but with errors.
  - The true output will not always be equal to the desired output.
  - Given input $x$, the actual output is $\boldsymbol{w}^T x$, the desired output is $y(x)$.
  - $\boldsymbol{w}$ is computed to minimize the error between the acutual output and the desired output.




© Copyright National University of Singapore. All Rights Reserved.

# Limitations of Linear Classifiers?

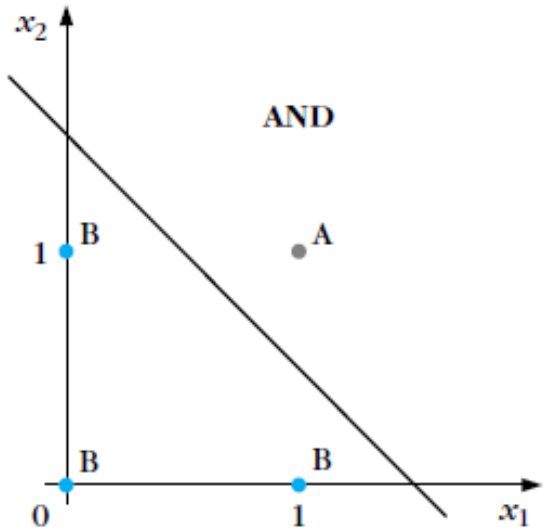
- Focus from the performance perspective:
  - Linear separable – ultimately 0 errors, what about linearly inseparable?
  - The inseparable data should always be the minority.
  - Are there cases where even when we design a linear classifier, in an optimal way, would result in poor performance – majority erroneous classification?
- One example: The XOR problem.

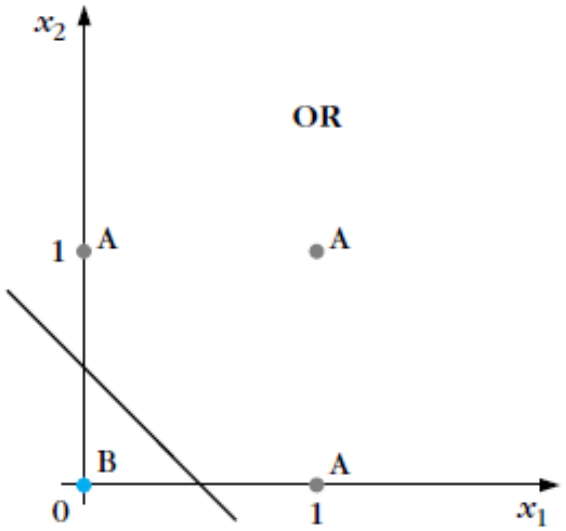


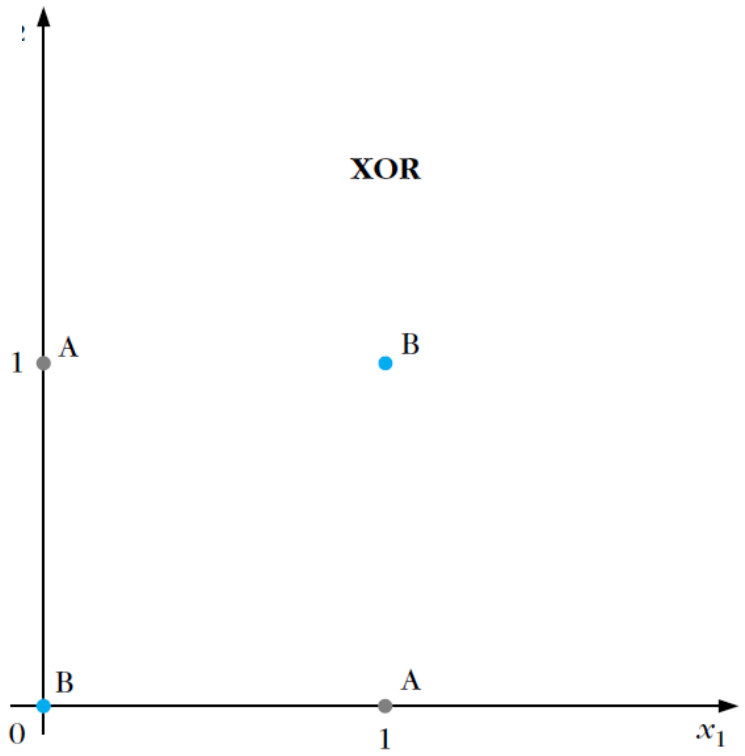

© Copyright National University of Singapore. All Rights Reserved.

# The XOR Problem

| $x_1$ | $x_2$ | XOR | Class |
|---|---|---|---|
| 0 | 0 | 0 | B |
| 0 | 1 | 1 | A |
| 1 | 0 | 1 | A |
| 1 | 1 | 0 | B |



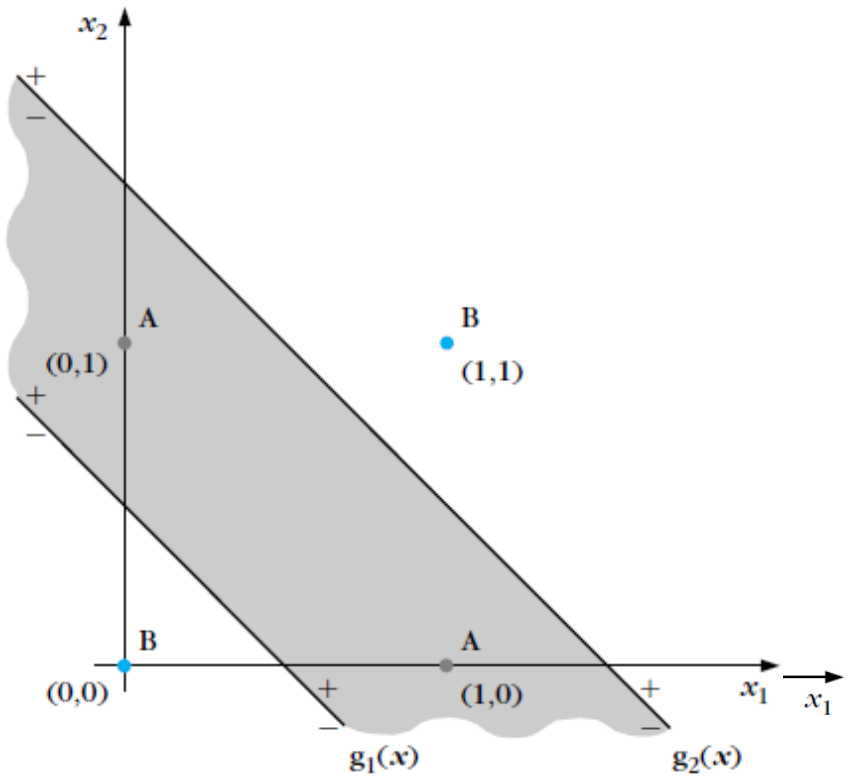

- Compare with other Boolean functions: AND and OR
- Both are linearly separable.
- The question now: how to tackle the XOR problem and extend further to other nonlinearly separable classification cases?

© Copyright National University of Singapore. All Rights Reserved.

# The Two-Layer Perceptron

- How to separate the two classes in the XOR problem? Any intuitive idea?
  - Think of leveraging a variant of the perceptron?
- A first thought can be: drawing 2 lines instead of one line!

- Attack the problem in two successive phases:
- Calculate the position of a feature vector $x$ with respect to each of the two decision lines.
- Combine the results of the previous phase and we find the position of $x$ with respect to both lines, i.e., outside or inside the shaded area.

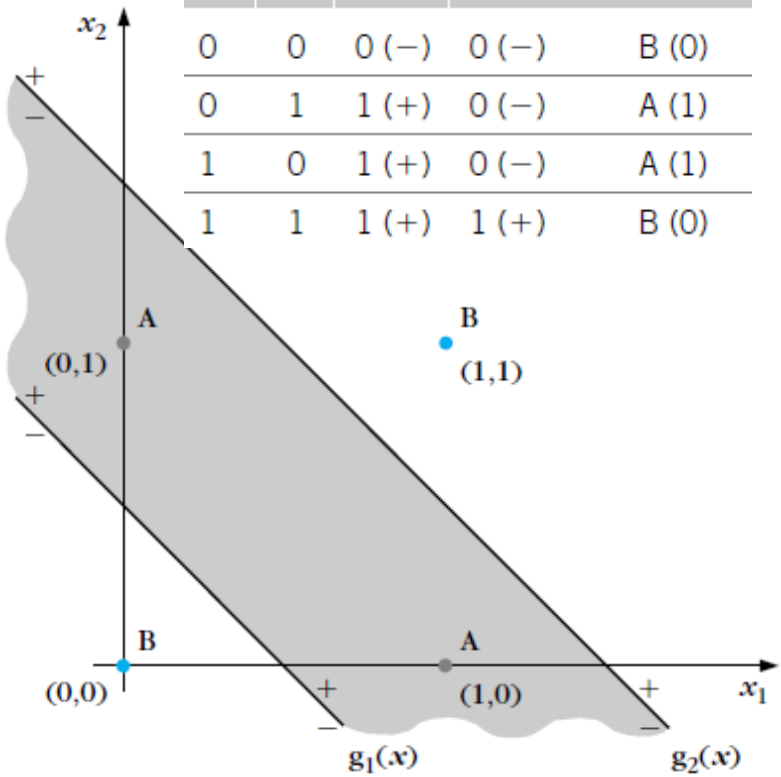



© Copyright National University of Singapore. All Rights Reserved.

# The Two-Layer Perceptron

- How to realize such a successive phased solution?
  - First, the two decision lines, or hyperplanes, $g_1$ and $g_2$, achieved with the adoption of 2 perceptrons:
  - Outputs $y_i = f(g_i(x))$ where $f$ is the activation function, a common one being the step function.
  - From another point of view:
  - Computations during the 1st phase perform a **mapping** of the input vector $x$ to a new one $y$ (usually what we call a **feature vector**).
  - The decision during the second phase is now based on the **transformed data**.

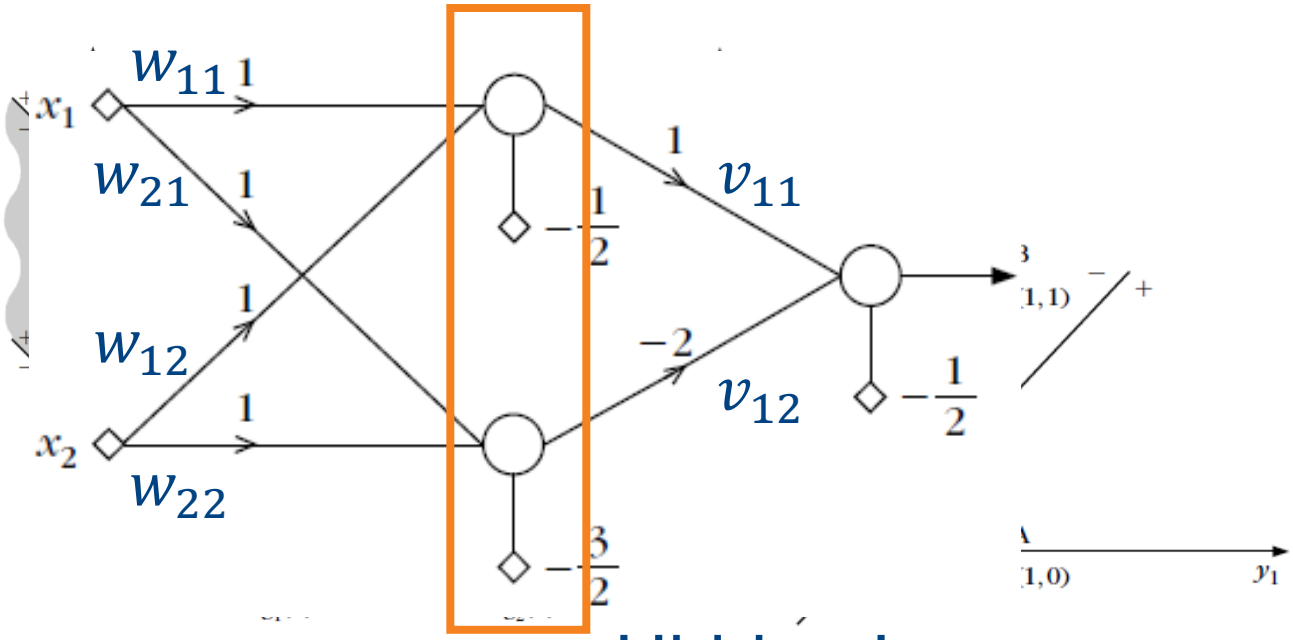
| 1st Phase | | | | |
|---|---|---|---|---|
| $x_1$ | $x_2$ | $y_1$ | $y_2$ | 2nd Phase |
| 0 | 0 | 0 (−) | 0 (−) | B (0) |
| 0 | 1 | 1 (+) | 0 (−) | A (1) |
| 1 | 0 | 1 (+) | 0 (−) | A (1) |
| 1 | 1 | 1 (+) | 1 (+) | B (0) |




© Copyright National University of Singapore. All Rights Reserved.

# The Two-Layer Perceptron

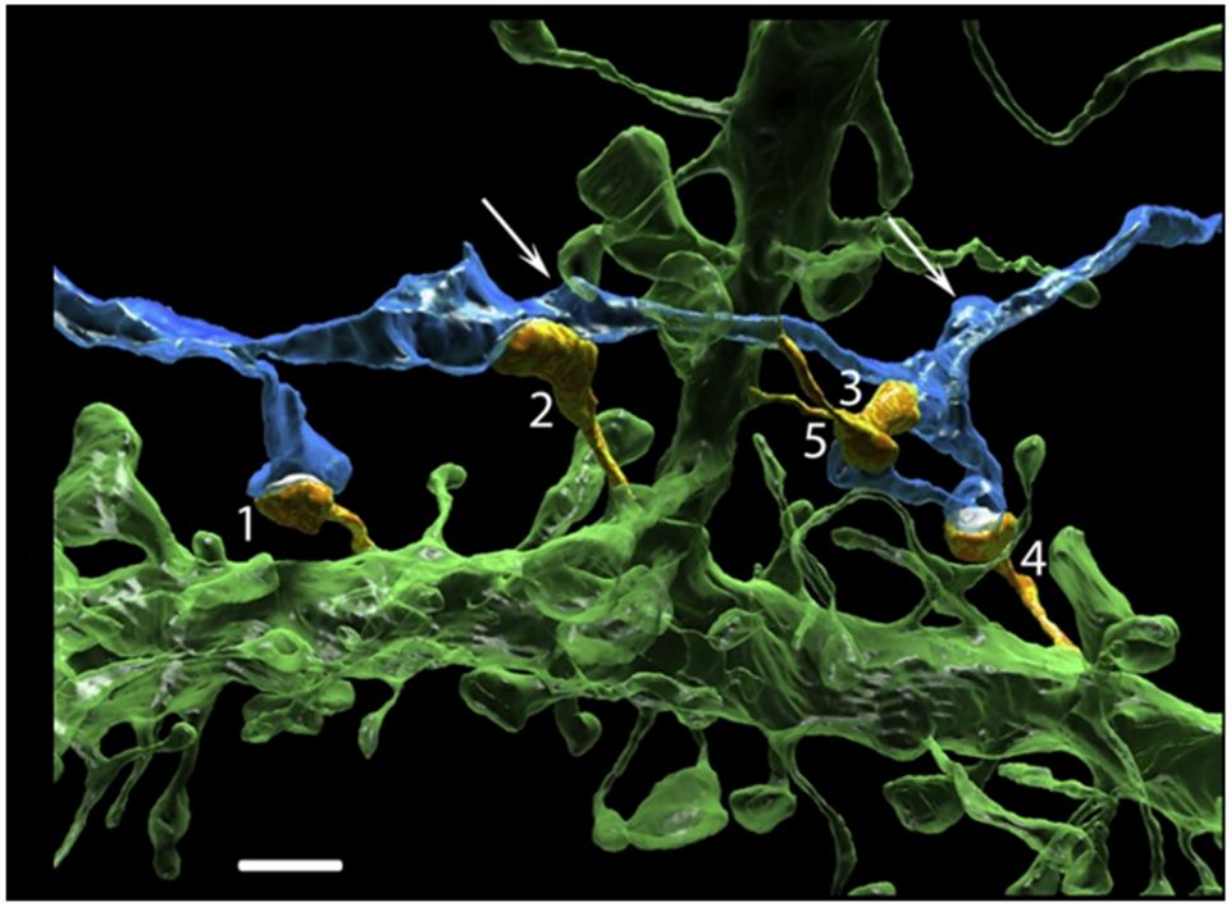
- IMPORTANT: the mapping of the 1st phase transforms the **nonlinearly separable** problem to a **linearly separable** one.
- The realization: through a multilayer structure – a generalization of the perceptron: a two-layer perceptron / two-layer feedforward neural network.

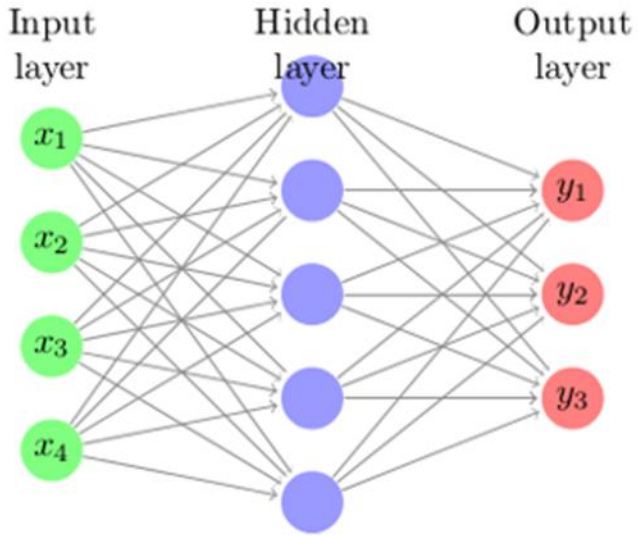



Hidden Layer

© Copyright National University of Singapore. All Rights Reserved.

# The Biological Inspiration Behind




© Copyright National University of Singapore. All Rights Reserved.

# Constraints of the Activation Function

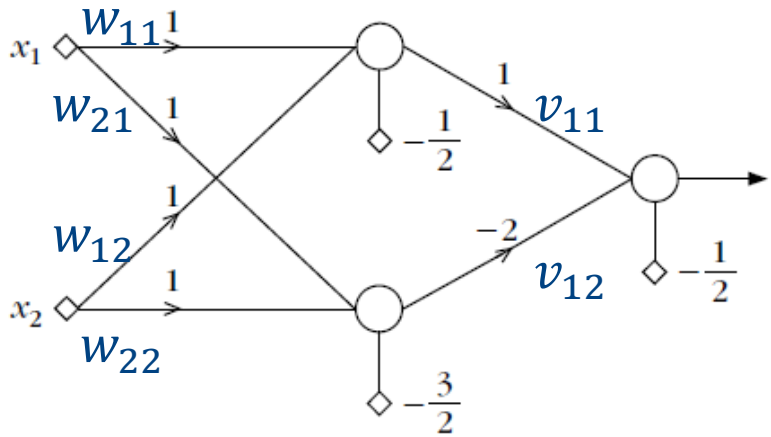
- Suppose input $x$ is in the $\mathbb{R}^l$ space and we have $p$ hidden layer neurons, final output denoted as $y$ and output from the hidden neuron as $z$.
- The mathematical expression of the outputs in the 2-layer perceptron is thus expressed as:

$$z_j = f\left(\sum_{i=1}^{l} w_{ji}x_i\right) = f\left(w_j^T x\right), y_k = f\left(\sum_{j=1}^{p} v_{kj}z_j\right) = f\left(v_k^T z\right)$$

- Which means that

$$y = f(V^T z) = f[V^T f(W^T x)]$$

- Question: any limitations for the activation function $f$?



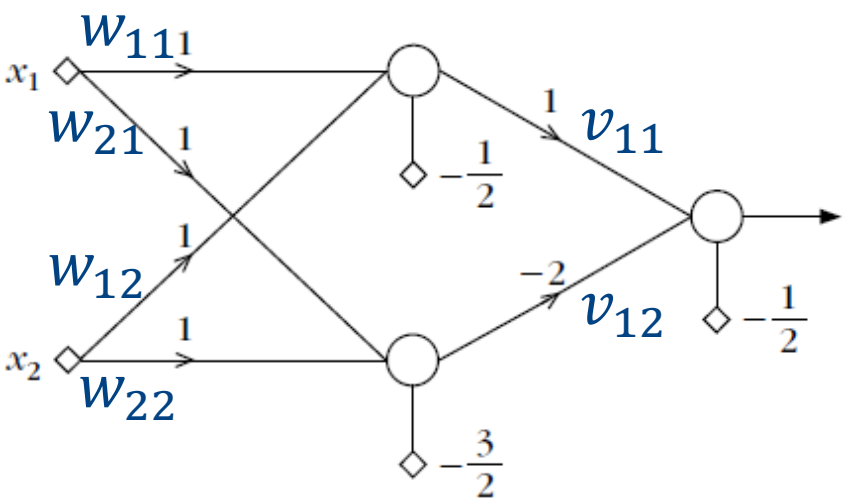

© Copyright National University of Singapore. All Rights Reserved.

# Constraints of the Activation Function

- Assume that the activation function $f$ is a linear function (a simple example: $f(r) = r$.
- Then the output can be computed as:

$$y = f(V^T z) = f[V^T f(W^T x)] = V^T W^T x = (WV)^T x = U^T x$$

- What does this imply?
  - The 2-layer perceptron will be **just the same** as the basic perceptron!
  - To design the 2-layer perceptron, it is important that the activation function MUST be a **non-linear** function, at least for hidden neurons.

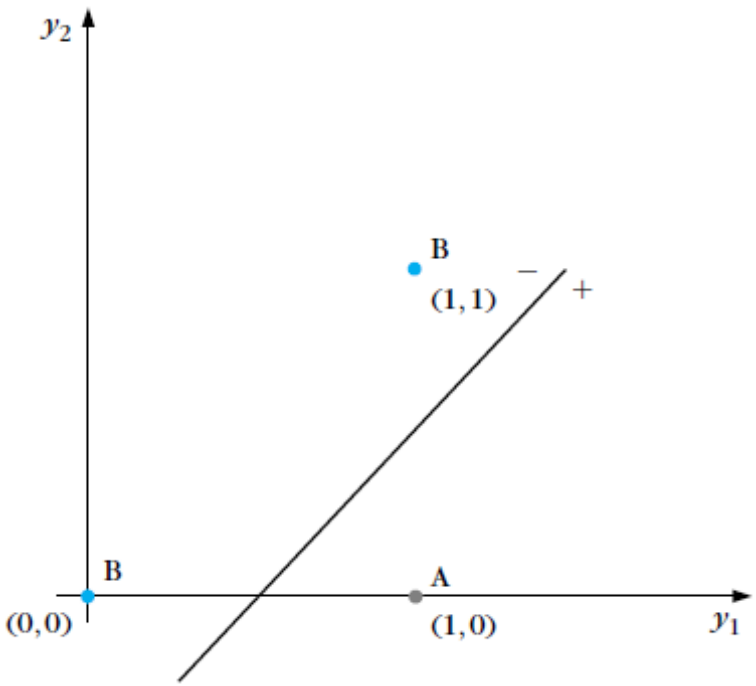



© Copyright National University of Singapore. All Rights Reserved.

# Capabilities of the Two-Layer Perceptron

- The action of neurons of the **hidden layer** is a **mapping** of the input space $x$ onto the vertices of a square of unit side length in the 2d space.
- Generally: consider input vectors $x \in \mathbb{R}^l$, and a two-layer perceptron with $p$ neurons in the hidden layer.
  - The mapping of the input space performed by the hidden layer is now onto the vertices of the hypercube of unit side length in the $p$-dim space ($H_p$), as

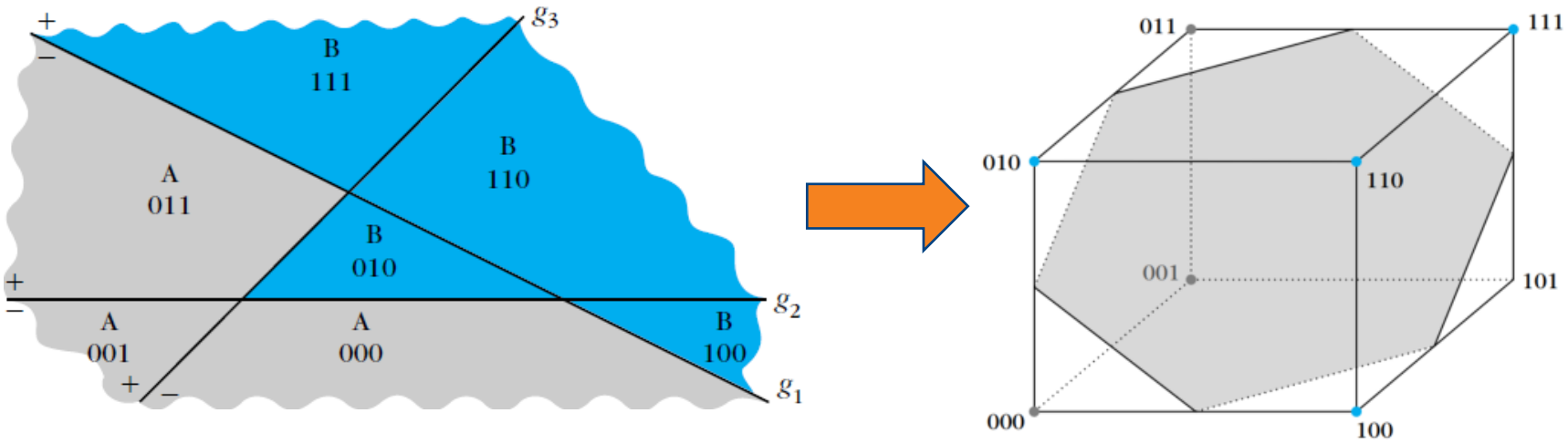
    $$H_p = \{[y_1, \dots, y_p]^T \in \mathbb{R}^p, y_i \in [0,1], i \in [1,p]\}$$
  - The mapping of the input space onto the vertices of the hypercube is via the creation of $p$ hyperplanes. Each of which is created by a neuron in the hidden layer.




© Copyright National University of Singapore. All Rights Reserved.

# Capabilities of the Two-Layer Perceptron

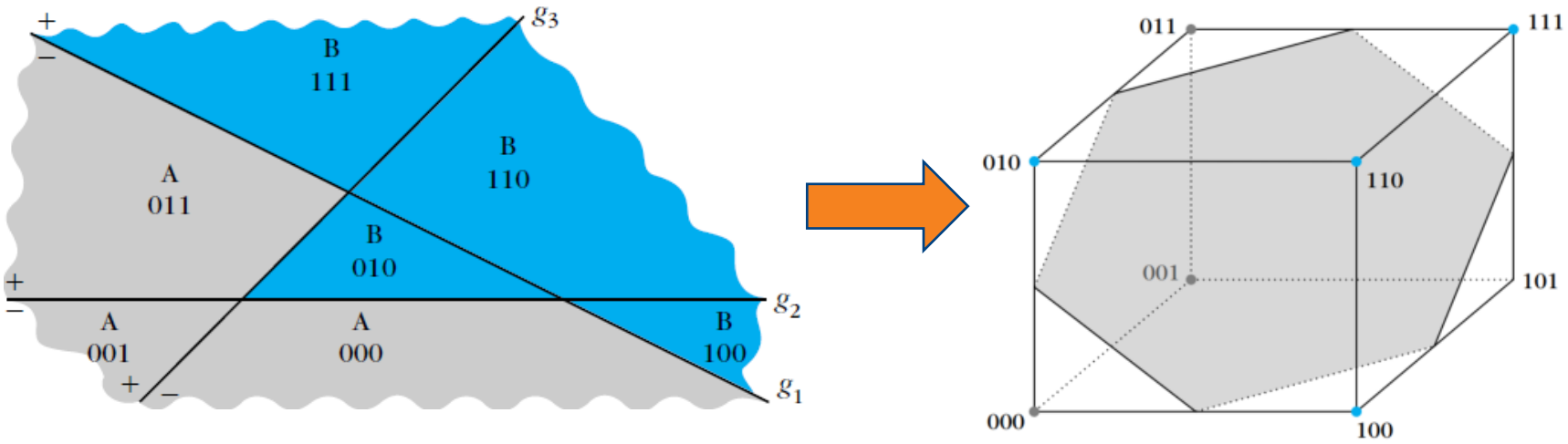
- An example of a 3-neuron hidden layer – i.e., 3 hyperplanes.
  - Each region defined by the intersections of the hyperplanes corresponds to a vertex of the unit 3-dim hypercube, depending on its position w.r.t. each of the hyperplanes.
  - Hidden layer neurons divides the input $l$-dim space into polyhedra, formed by hyperplane intersections.
  - All vectors located within one of these polyhedral regions are mapped onto a specific vertex of the unit $H_p$ hypercube.




© Copyright National University of Singapore. All Rights Reserved.

# Capabilities of the Two-Layer Perceptron

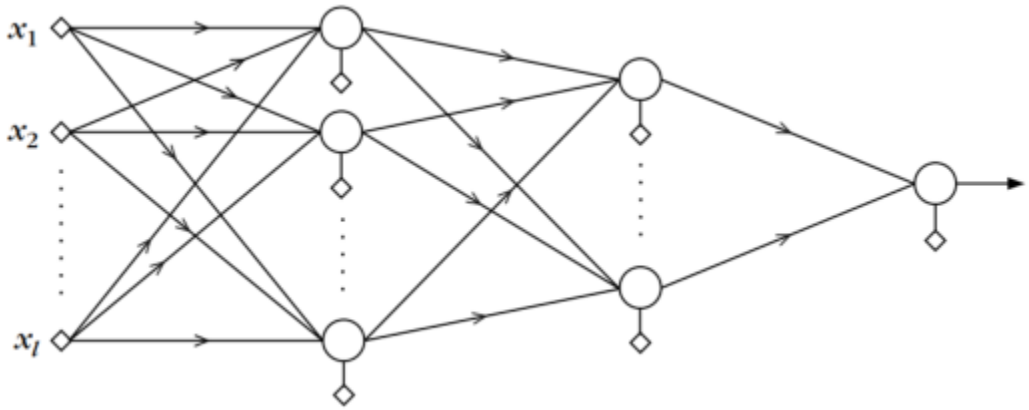
- An example of a 3-neuron hidden layer – i.e., 3 hyperplanes.
  - **Output** neuron realizes another hyperplane, separates the hypercube into 2 parts.
  - A 2-layer perceptron (a single hidden layer) can separate classes consisting of unions of polyhedral regions, but **NOT** any union of such regions – depends on the relative positions of the vertices of $H_p$, where the classes are mapped, and on whether they are linearly separable.




© Copyright National University of Singapore. All Rights Reserved.

# Further: Multi (Three)-Layer Perceptron (MLP)

- The inability of the 2-layer perceptrons to separate classes from any union of polyhedral regions is similar to the problem where the basic perceptron cannot deal with the XOR problem.
- Can we opt for a similar solution?
- Increase the complexity – constructing a three-layer perceptron.
- Leave to think: why would the increase of one layer of neurons enable the multi-layer perceptron to separate classes resulting from **any** union of polyhedral regions?




© Copyright National University of Singapore. All Rights Reserved.

# Recall: The Perceptron Algorithm

$$\boldsymbol{w}(t+1) = \boldsymbol{w}(t) - \rho_t \frac{\partial J(\boldsymbol{w})}{\partial \boldsymbol{w}}\Big|_{\boldsymbol{w}=\boldsymbol{w}(t)}, \text{where } J(w) = \sum_{x \in M} (\delta_x \boldsymbol{w}^T x)$$

- At the points where the gradient is valid, we have $\frac{\partial J(w)}{\partial w} = \sum_{x \in M} \delta_x x$. The optimization function is thus: $\boldsymbol{w}(t+1) = \boldsymbol{w}(t) - \rho_t \sum_{x \in M} \delta_x x$.
- This is the perceptron algorithm, with an arbitrary starting point $\boldsymbol{w}(0)$.

- Choose $\boldsymbol{w}(0)$ randomly
- Choose $\rho_0$
- $t = 0$
- Repeat
  - $M = \emptyset$
  - For $i = 1$ to $N$
    - If $\delta_{x_i} \boldsymbol{w}(t)^T \boldsymbol{x}_i \geq 0$ then $M = M \cup \{\boldsymbol{x}_i\}$
  - End {For}
  - $\boldsymbol{w}(t+1) = \boldsymbol{w}(t) - \rho_t \sum_{x \in M} \delta_x x$.
  - Adjust $\rho_t$
  - $t = t + 1$
- Until $M = \emptyset$

© Copyright National University of Singapore. All Rights Reserved.

# Adjusting w.r.t. Cost Function

- The core idea of the perceptron algorithm: **adjust** the synaptic params (weights) to minimize a **cost function** of its output.

$$\boldsymbol{w}(t+1) = \boldsymbol{w}(t) - \rho_t \frac{\partial J(\boldsymbol{w})}{\partial \boldsymbol{w}}\bigg|_{\boldsymbol{w}=\boldsymbol{w}(t)}$$

- What do we need to know?
  - The cost function (to choose, discuss later);
  - The differentiation of the cost function w.r.t. the weights.
  - Problem?
- Currently, our MLPs are developed around the McCulloch-Pitts neurons – the ***hard limiter*** step function as activation function!

© Copyright National University of Singapore. All Rights Reserved.

# Continuous Activation Function

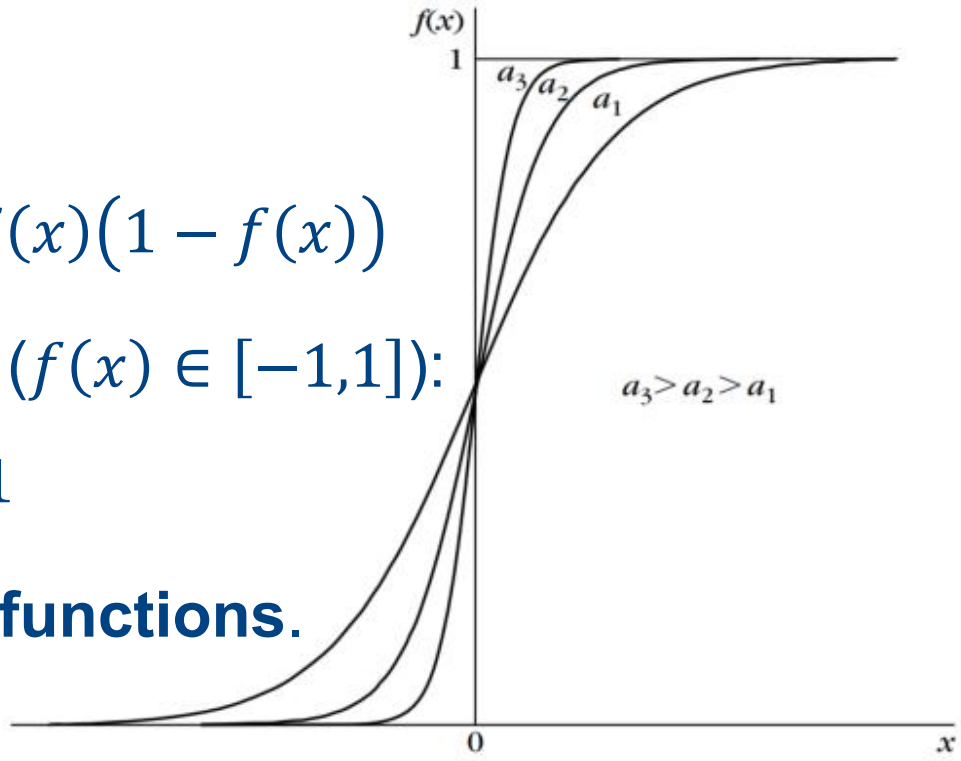
- The problem: not differentiable at some input ($x = 0$)!
- Possible solution: finding a **continuous** differentiable activation function that behaves similar to the step function.
  - The family of sigmoid functions;
  - Representative: the logistic function:

$$f(x) = \frac{1}{1 + \exp(-ax)}, \text{with } f'(x) = af(x)\big(1 - f(x)\big)$$

  - Variant that is antisymmetric w.r.t. the origin ($f(x) \in [-1,1]$):

$$f(x) = \frac{2}{1 + \exp(-ax)} - 1$$

  - These funcs are also known as **squashing functions**.




© Copyright National University of Singapore. All Rights Reserved.

# Note: Indifferentiable Function

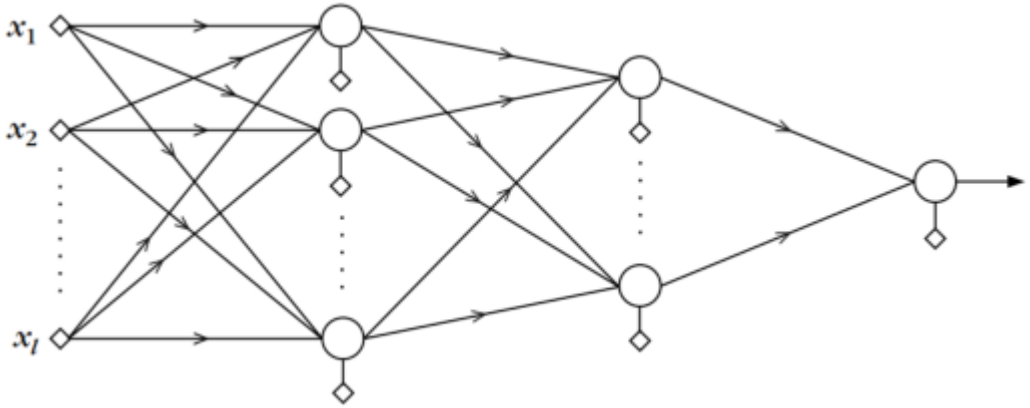
- In actual implementation, it is actually quite common to adopt activation functions that are not differentiable at some input.
- In such cases, it is necessary to self-define the gradient in the coding (example on the right).

```
class _SimpleConsensus(torch.autograd.Function):
        """Simplest segmental consensus module"""

        '''def __init__(self,
                                consensus_type='avg',
                                dim=1):
                super(_SimpleConsensus, self).__init__()
                assert consensus_type in ['avg']
                self.consensus_type = consensus_type
                self.dim = dim
                self.shape = None'''

        @staticmethod
        def forward(ctx, x, consensus_type='avg', dim=1):
                ctx.shape = x.size()
                ctx.consensus_type = consensus_type
                ctx.dim = dim
                if ctx.consensus_type == 'avg':
                        output = x.mean(dim=ctx.dim, keepdim=True)
                else:
                        output = None
                return output

        @staticmethod
        def backward(ctx, grad_output):
                if ctx.consensus_type == 'avg':
                        grad_in = grad_output.expand(ctx.shape)
                else:
                        grad_in = None
                return grad_in, None, None
```

© Copyright National University of Singapore. All Rights Reserved.

# Training the MLP

- Assume a multi-layer neural architecture with the sigmoid activation. The goal: an iterative training algorithm that computes the synaptic weights of the network so that the cost function (chosen) is minimized).
- For generalization, assume $L$ layers of neuron, with $k_0 = l$ nodes in the input layer and $k_r$ neurons in the $r$-th layer. Assume a total of $N$ training pairs $[y(i), x(i)]$. The output is a $k_L$-dim output vector. The output from the network of input $x(i)$ is $\hat{y}(i)$, which may differ from the desired value $y(i)$.
- The synaptic weights are computed such that an appropriate cost function $J$, which is dependent on the values $y(i)$ and $\hat{y}(i), i = 1, \ldots, N$, is minimized in an iterative manner.




© Copyright National University of Singapore. All Rights Reserved.

# Training the MLP: Backpropagation

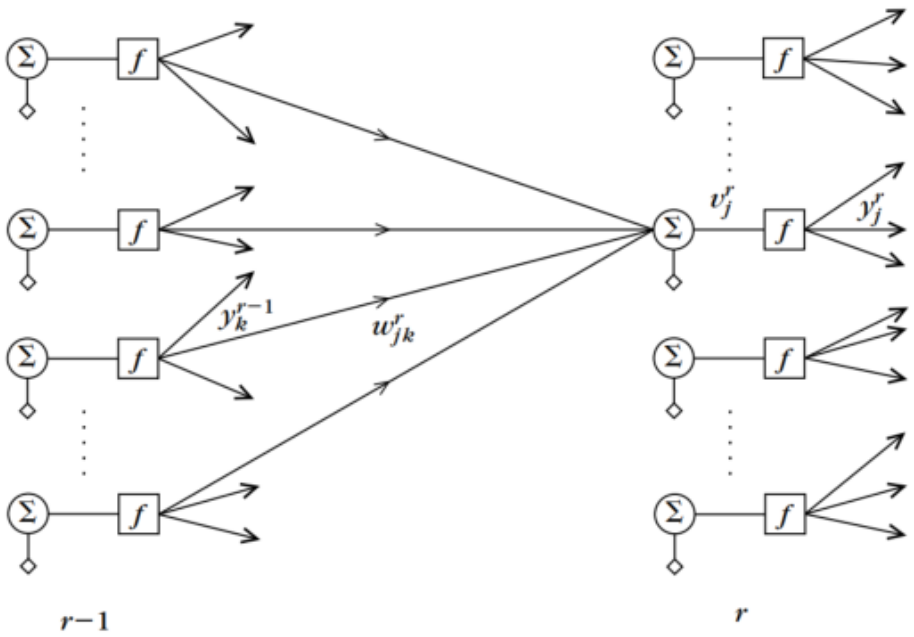
- The whole process can be literally viewed as extending the propagation algorithm across the different layers of neurons:
  - Still, follow the spirit of gradient descent.
  - Define $w_j^r$ be the $(k_{r-1}+1)$-dim weight vector of the $j$-th neuron in the $r$-th layer.
  - The basic iteration step is of the form:

$$w_j^r(new) = w_j^r(old) + \Delta w_j^r, \Delta w_j^r = -\mu \frac{\partial J}{\partial w_j^r}$$

  - Generally, the cost function can be formulated as:

$$J = \sum_{i=1}^{N} \mathcal{E}(i)$$

  - Thus, the cost function $J$ is the sum of the $N$ values that function $\mathcal{E}$ takes for each of the training pairs.




© Copyright National University of Singapore. All Rights Reserved.

# Training the MLP: Backpropagation

- Next, we detail on the computation of the gradients:
  - If $y_k^{r-1}(i)$ is the output of the $k$-th neuron in the $(r-1)$-th layer for the $i$-th training pair, the argument for the activation function $f$ w.r.t. the output neuron ($y_j^r$) will be:

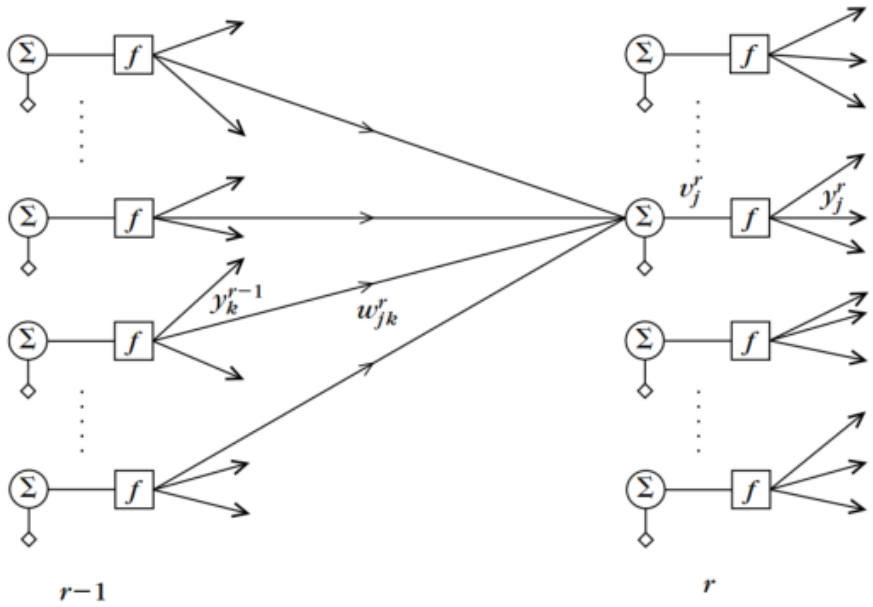
$$v_j^r = \sum_{k=0}^{k_{r-1}} w_{jk}^r \, y_k^{r-1}(i)$$

  - Apparently, the dependence of $\mathcal{E}(i)$ on $w_j^r$ passes through $v_j^r$. Leverage on the chain rule, we thus have:

$$\frac{\partial \mathcal{E}(i)}{\partial w_j^r} = \frac{\partial \mathcal{E}(i)}{\partial v_j^r(i)} \frac{\partial v_j^r(i)}{\partial w_j^r}$$

  - Observe that $\frac{\partial}{\partial w_j^r} v_j^r(i) \equiv y^{r-1}(i)$. Define $\frac{\partial \mathcal{E}(i)}{\partial v_j^r(i)} = \delta_j^r(i)$:

$$\Delta w_j^r = -\mu \frac{\partial J}{\partial w_j^r} = -\mu \sum_{i=1}^{N} \delta_j^r(i) y^{r-1}(i)$$



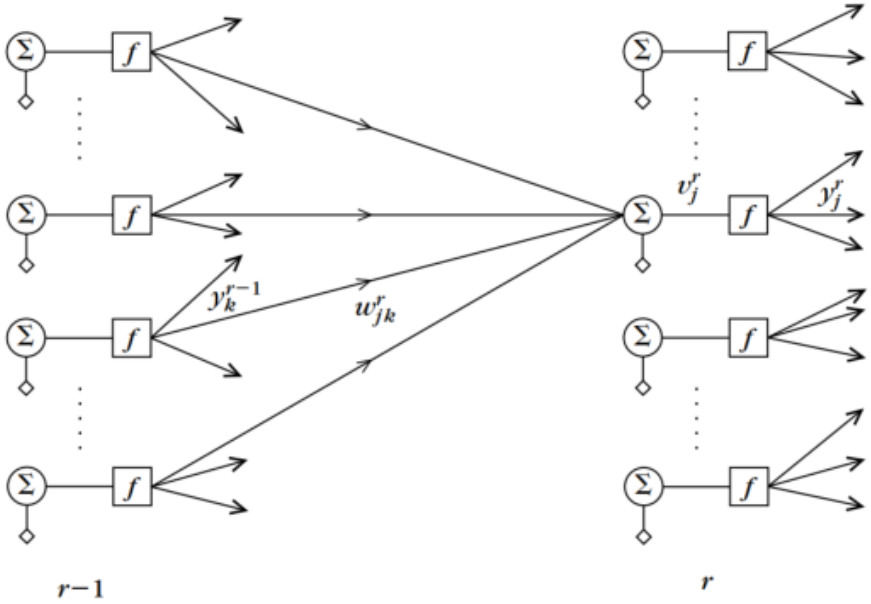

© Copyright National University of Singapore. All Rights Reserved.

# Training the MLP: Backpropagation

- Last, we detail on the computation of $\delta_j^r(i)$:
  - The computations start from $r = L$ and propagate **backward** for $r = L_1, L_2, \dots, 1$. I.e., the computation start from the last layer and propagate towards the first layer.
  - This is the reason the algorithm derived is known as the ***backpropagation*** algorithm.
  - For $r = L$ (last layer): $\delta_j^L(i) = \frac{\partial \mathcal{E}(i)}{\partial v_j^L(i)}$. A common $\mathcal{E}$ can be the sum of squared errors in output neurons. Thus:

$$\mathcal{E}(i) \equiv \frac{1}{2}\sum_{m=1}^{k_L} e_m^2(i) \equiv \frac{1}{2}\sum_{m=1}^{k_L}\left(f\left(v_m^L(i)\right) - y_m(i)\right)^2$$

  - Hence $\delta_j^L(i) = e_j(i) f'\left(v_j^L(i)\right)$, $f'(\cdot)$ is the derivative of $f$. Thus, in the last layer, the dependence of $\mathcal{E}(i)$ on $v_j^L(i)$ is explicit, and the computation of the derivative is straightforward.




© Copyright National University of Singapore. All Rights Reserved.

# Training the MLP: Backpropagation

- Last, we detail on the computation of $\delta_j^r(i)$:
  - For $r < L$, due to the successive dependence among the layers, the value of $v_j^{r-1}(i)$ influences all $v_k^r(i), k = 1,2,\dots,k_r$ of the next layer. Employing the chain rule, we have:

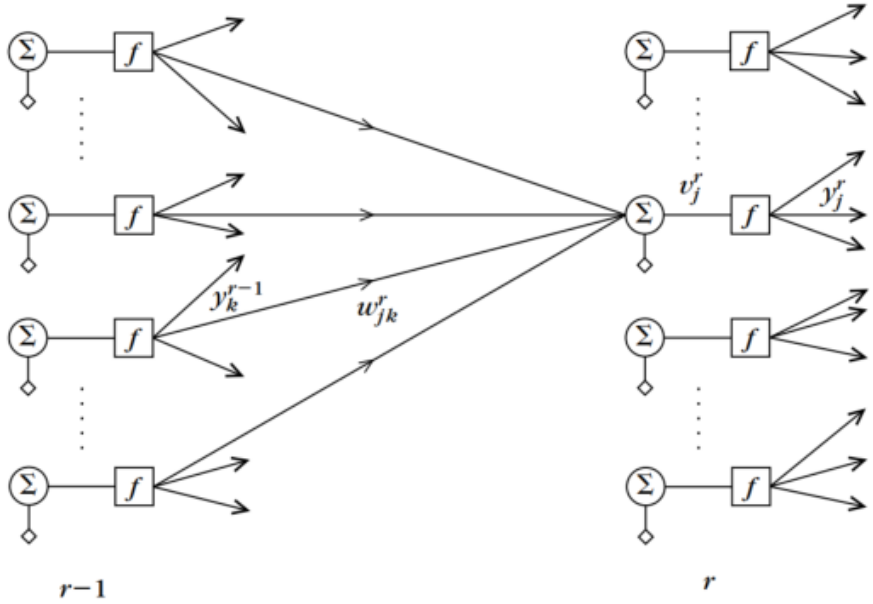
$$\frac{\partial \mathcal{E}(i)}{\partial v_j^{r-1}(i)} = \sum_{k=1}^{k_r} \frac{\partial \mathcal{E}(i)}{\partial v_k^r(i)} \frac{\partial v_k^r(i)}{\partial v_j^{r-1}(i)} \Rightarrow \delta_j^{r-1}(i) = \sum_{k=1}^{k_r} \delta_k^r(i) \frac{\partial v_k^r(i)}{\partial v_j^{r-1}(i)}$$

  - The above formula is further computed by the fact that:

$$\frac{\partial v_k^r(i)}{\partial v_j^{r-1}(i)} = \frac{\partial \left[\sum_{m=0}^{k_{r-1}} w_{km}^r y_m^{r-1}(i)\right]}{\partial v_j^{r-1}(i)}, y_m^{r-1}(i) = f(v_m^{r-1}(i))$$

$$\frac{\partial v_k^r(i)}{\partial v_j^{r-1}(i)} = w_{kj}^r f'\left(v_j^{r-1}(i)\right)$$

$$\delta_j^{r-1}(i) = \left[\sum_{k=1}^{k_r} \delta_k^r(i) w_{kj}^r\right] f'\left(v_j^{r-1}(i)\right) = e_j^{r-1}(i) f'\left(v_j^{r-1}(i)\right)$$




© Copyright National University of Singapore. All Rights Reserved.

# Training the MLP: Backpropagation

- The overall Backpropagation Algorithm summarized as:
  - 1. Initialization: initialize all the weights with small random values from a pseudo-random sequence generator.
  - 2. Forward computations: for each of the training feature vector, compute all $v_j^r(i)$ and $y_j^r(i) = f\left(v_j^r(i)\right), i \in [1, N], j \in [1, k_r], r \in [1, L]$. Compute the cost function for the current estimate of weights.
  - 3. Backward computations: for each $i$ and $j$ compute $\delta_j^L(i)$ and in the sequel compute $\delta_j^{r-1}(i)$ for $r = L, L-1, \dots, 2$ and $j = 1,2, \dots, k_r$.
  - 4. Update the weights with $w_j^r(new) = w_j^r(old) + \Delta w_j^r$ and $\Delta w_j^r = -\mu \sum_{i=1}^{N} \delta_j^r(i) y^{r-1}(i)$.

© Copyright National University of Singapore. All Rights Reserved.

# Beyond the Basic MLP

- The MLP we discussed (viewed as a basic MLP network) with sigmoidal activation function produces a global hyperplane.
- Discuss: what is the pros and cons of such hyperplane?
- Is such hyperplane universally applicable?
- In the nervous system of biological organisms, there is evidence that responses of some neurons are "**local**" or tuned to some region of input space.
  - An example is the orientation sensitive cells of the visual cortex, whose response is sensitive to local region in the retina.

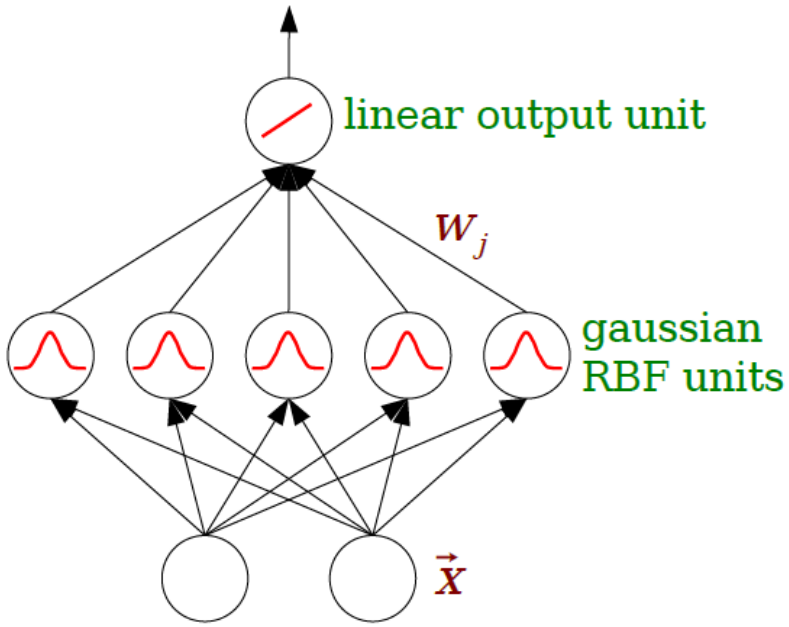
© Copyright National University of Singapore. All Rights Reserved.

# Radial Basis Function (RBF) Networks

- To realize such locality behavior, introduce the Radial Basis Function (RBF) network.
- Observe: a similar structure compared to MLP, difference?

- The function of the hidden layer:
  - MLP: sigmoidal activation function – hyperplane
  - RBF: radial basis function – hyper-ellipse
  - For each of the RBF unit (hidden layer neuron), the operation is formulated as:

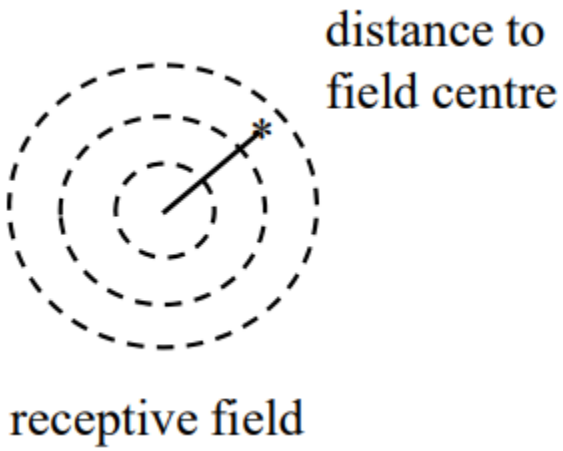
$$d_i = \| x - c_i \|$$
$$o_i = g(d_i) = g(\| x - c_i \|)$$

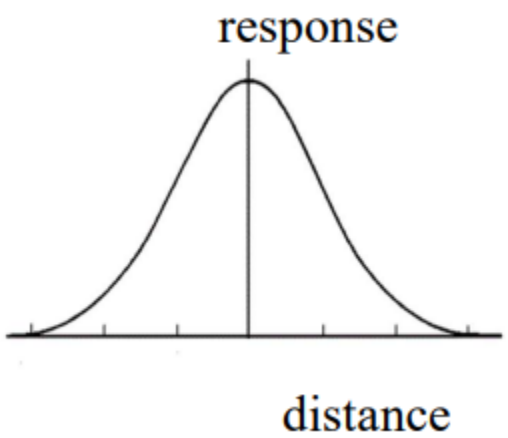


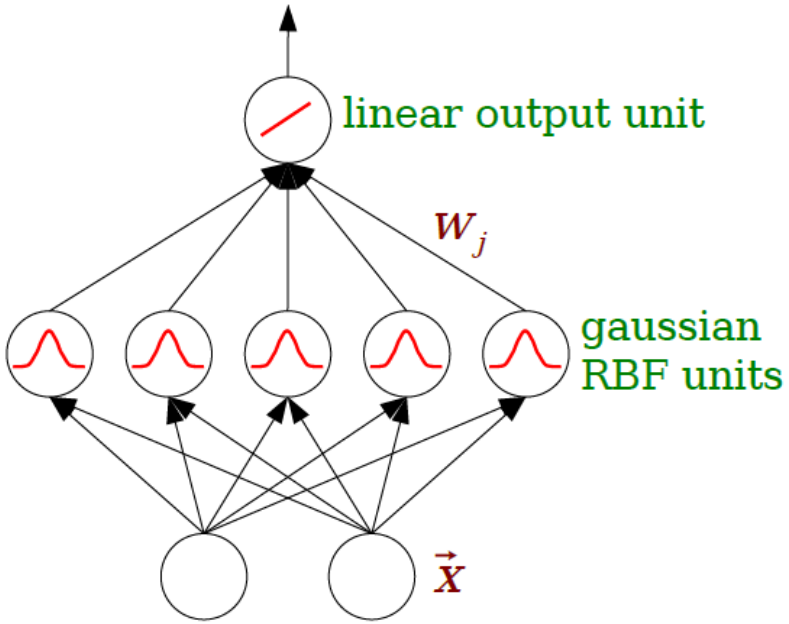

© Copyright National University of Singapore. All Rights Reserved.

# Understanding the RBF Network

- RBF networks emulate behavior of certain **biological** networks:
- The hidden layer consists of the locally tuned or locally sensitive neurons, whose response (output) is localized and decays as a function of the distance from the input to the center of neuron's receptive field.



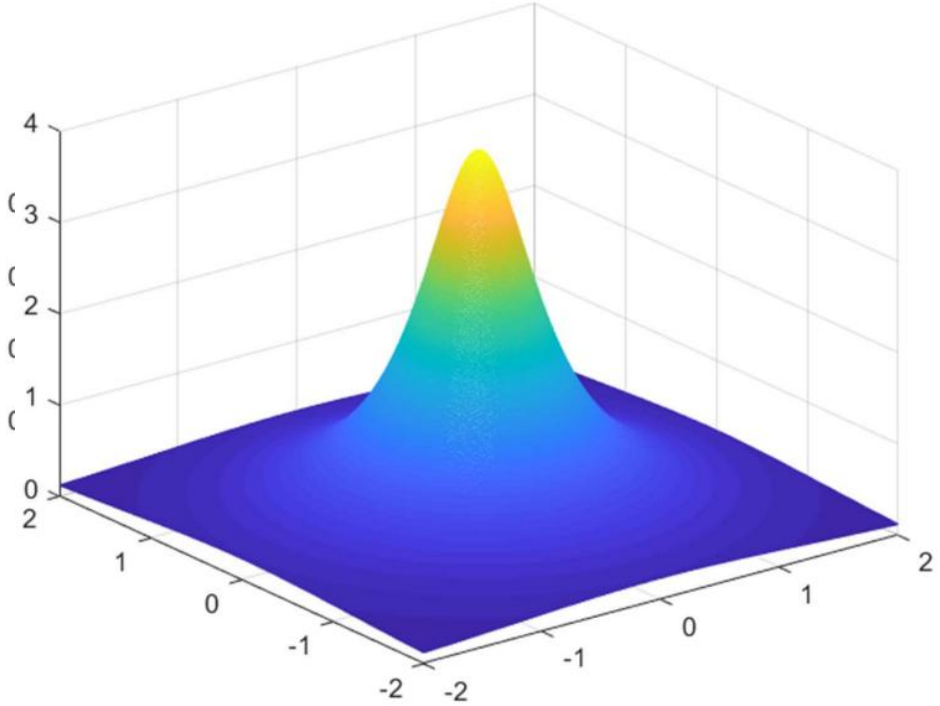




© Copyright National University of Singapore. All Rights Reserved.

# RBF Networks: The RBFs

- One of the core of the RBF network is the choice of the RBF (i.e., function $g(\cdot)$ of the hidden layer neuron).
- Common choice of RBF include:
  - Gaussian function:
  - $g(d_i) = \exp\left(-\frac{d_i^2}{2\sigma^2}\right)$
  - Inverse multi-quadratic function:
  - $g(d_i) = 1/\sqrt{d_i^2 + \sigma^2}$

inverse multi-quadratic
σ=0.5




© Copyright National University of Singapore. All Rights Reserved.

# Training an RBF Network

- The output layer neuron is a linear combiner, i.e., weighted sum of the hidden layer neuron outputs. Assume the Gaussian function is used for the hidden layer neurons, the output is:

$$f(x) = \sum_{j=1}^{m} w_j o_j(x) = \sum_{j=1}^{m} w_j \exp\left(-\frac{\| x - c_j \|^2}{2\sigma^2}\right)$$

- What are the parameters that determine the RBF network?
  - The center vectors $c_j$;
  - The weights $w_j$ corresponding to each RBF hidden neuron;
  - The width of the basis function $\sigma$.

© Copyright National University of Singapore. All Rights Reserved.

# Training an RBF Network: a Hybrid Process

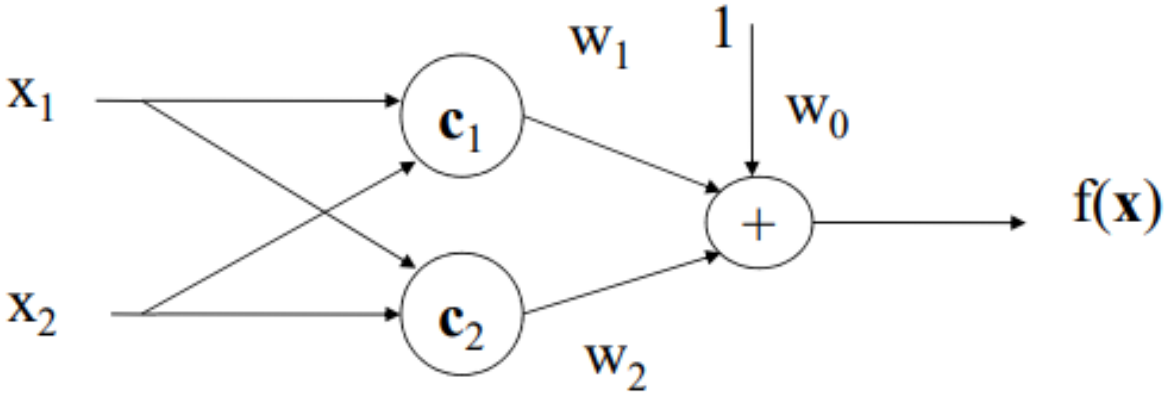
- There are two steps for determining the RBF:
  - 1. Determine the RBF centers.
  - 2. Estimation of the weights.
- Usually, the overall training process is a **hybrid** process:
  - The RBF centers are determined in an **unsupervised** manner;
  - The weight estimation is computed in a **supervised** manner.
- For determine the RBF centers, possible methods are:
  - Random selection;
  - Sequential forward selection (merge from individual neurons);
  - Clustering (e.g., $k$-means, to be discussed in Second half of course!)

© Copyright National University of Singapore. All Rights Reserved.

# Training an RBF Network: a Hybrid Process

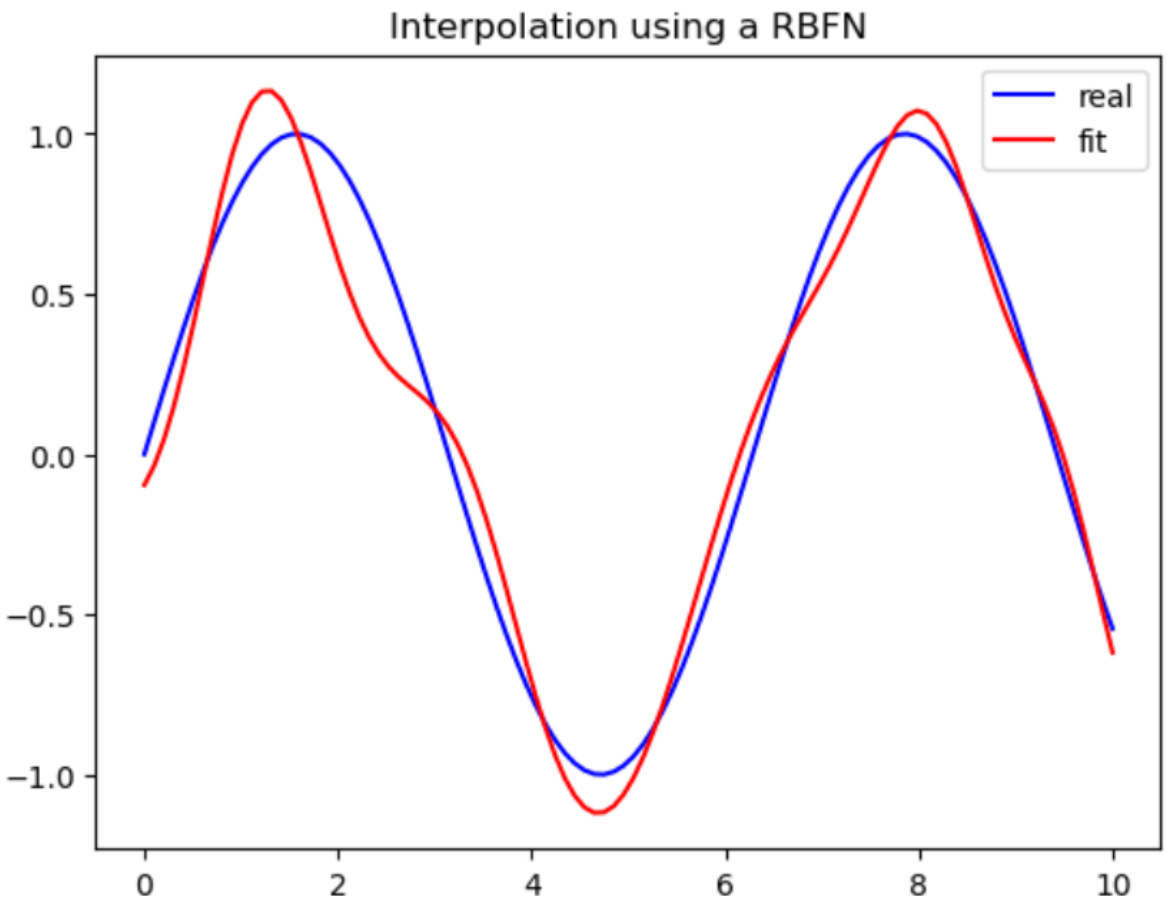
- For the weight estimation, recall LS estimation (least squares) in linear network – the link between the hidden and output layer neurons are linear: $J(\boldsymbol{w}) = \sum_i^N \left(y_i - x_i^T \boldsymbol{w}\right)^2 \equiv \sum_i^N e_i^2$
- Note that the input now is not the original input, but the output from the radial basis function (after the hidden layer neuron).
- Try it out with Gaussian RBF: if the center neurons are $c_1 = [1\ 1]^T$ and $c_2 = [0\ 0]^T$, with width $\sigma = 0.707$, what would be the estimated weights?

| $x_1$ | $x_2$ | XOR | Class |
|---|---|---|---|
| 0 | 0 | 0 | B |
| 0 | 1 | 1 | A |
| 1 | 0 | 1 | A |
| 1 | 1 | 0 | B |




© Copyright National University of Singapore. All Rights Reserved.

# RBF Network: An Example with CV




© Copyright National University of Singapore. All Rights Reserved.

# End of Week 5

© Copyright National University of Singapore. All Rights Reserved.